

Trả lời câu hỏi cuối bài:

Bài 1: Kiến trúc tổng quan

1. 8086 là vi điều khiển(MCU-microcontroller) hay vi xử lý(CPU/MPU-Microprocessor)?
 - Là 1 con vi xử lý (MPU - Microprocessor).
Vi bên trong chip chỉ có nhân xử lý (CU, ALU, thanh ghi), không tích hợp sẵn bộ nhớ RAM/ROM, cổng I/O hay timer như vi điều khiển (8051, PIC, AVR, STM32). Muốn chạy phải ghép thêm chip bên ngoài.
2. Kiến trúc 8086 gồm 3 phần chính gì?
 - Control Unit (CU), ALU (Arithmetic Logic Unit), Registers (thanh ghi).
 - CU: nạp lệnh, giải mã lệnh, phát tín hiệu điều khiển cho các bộ phận khác.
 - ALU: thực hiện phép số học (ADD, SUB, MUL, DIV), phép logic (AND, OR, XOR, NOT), dịch/quay bit và so sánh (CMP); kết quả tính xong sẽ cập nhật thanh ghi cờ.
 - Registers: ô nhớ siêu nhanh nằm trong CPU, chứa toán hạng, kết quả trung gian, địa chỉ và trạng thái.
3. Thanh ghi thực tế là gì? Chức năng của nó là gì?
 - Thanh ghi là ô nhớ nằm ngay bên trong CPU, dùng để chứa toán hạng cho ALU, kết quả trung gian, địa chỉ và trạng thái.
Khác với bộ nhớ RAM ở bốn điểm:
 - Nằm bên trong CPU nên tốc độ nhanh nhất hệ thống, truy cập không phải đi qua bus.
 - Không có địa chỉ bộ nhớ, được gọi bằng tên (AX, BX, CX, DX...).
 - Số lượng rất ít: 8086 chỉ có 14 thanh ghi.
 - Kích thước 16 bit; riêng AX, BX, CX, DX tách được thành hai nửa 8 bit (AH/AL, BH/BL...).
4. 8086 có thể chạy 1 mình không? Tại sao? Và làm sao để 8086 có thể chạy được?
 - Không. 8086 là vi xử lý, bên trong chỉ có CU, ALU và thanh ghi nên còn thiếu:
 - Bộ nhớ ROM/RAM để chứa chương trình và dữ liệu.
 - Mạch tạo xung clock: 8086 không tự tạo được xung nhịp, phải dùng chip 8284A.
 - Mạch chốt địa chỉ: bus AD0-AD15 là bus dồn kênh (vừa mang địa chỉ vừa mang dữ liệu), cần chốt 74LS373 tách địa chỉ và đệm 74LS245 cho dữ liệu.
 - Thiết bị vào/ra và nguồn nuôi +5V.Muốn chạy được phải ghép thành hệ vi xử lý: 8086 + 8284A + ROM + RAM + mạch chốt/đệm bus + bộ điều khiển bus 8288 (chế độ maximum) + chip ngoại vi (8255, 8259, 8253), nối với nhau qua ba bus: bus địa chỉ 20 đường, bus dữ liệu 16 đường và bus điều khiển.
5. 8086 là chip xử lý bao nhiêu bit ?
 - 8086 là chip xử lý 16 bit: thanh ghi đa dụng, ALU và bus dữ liệu đều rộng 16 bit.
Lưu ý: riêng bus địa chỉ có 20 đường nên đánh địa chỉ được $2^{20} = 1 \text{ MB}$, chứ không phải 64 KB. Đó chính là lý do phải dùng cơ chế segment:offset (Vật lý = Segment x 16 + Offset).
6. Trong 1 lần tính, thì Bộ cộng, trừ, nhân, chia của 8086 có thể thực hiện giữa 2 số lớn nhất là bao nhiêu?
 - Có thể thực hiện giữa 2 số lớn nhất là 65535 (số không dấu 16 bit); nếu tính số có dấu thì phạm vi là -32768 đến +32767.
Lưu ý: toán hạng đầu vào tối đa 16 bit nhưng kết quả có thể lớn hơn. Lệnh MUL 16 bit cho kết quả 32 bit chứa trong cặp DX:AX (DX giữ word cao, AX giữ word thấp); lệnh DIV lấy số bị chia 32 bit từ DX:AX, thương vào AX và số dư vào DX.
7. Chức năng của 8086 là gì
 - Chức năng của 8086 là thực thi chương trình, bằng cách lặp đi lặp lại chu trình Fetch - Decode - Execute:
 - Fetch (nạp lệnh): CU ghép CS:IP thành địa chỉ, đưa lên bus địa chỉ và nạp mã lệnh từ bộ nhớ về CPU; IP tự động tăng đúng số byte của lệnh vừa nạp.
 - Decode (giải mã): CU phân tích opcode để biết đó là lệnh gì, cần mấy toán hạng và lấy ở

đầu.

- Execute (thực thi): ALU tính toán hoặc dữ liệu được chuyển giữa thanh ghi và bộ nhớ; kết quả ghi vào toán hạng đích và thanh ghi cờ được cập nhật.

Chu trình này lặp hàng triệu lần mỗi giây và CPU không làm gì khác ngoài nó. CPU không "hiểu" chương trình: nó chỉ đọc từng byte, tra xem byte đó là opcode nào rồi thực hiện một thao tác rất đơn giản.

Bài 3: Các hệ đếm decimal, binary, hexa

8. Cho chương trình:

MOV AX, 1101b

MOV BX, 0Dh

MOV CX, 13

Hỏi: Kết quả trong 3 thanh ghi là như nhau hay khác nhau?

- Như nhau, cả ba thanh ghi đều bằng 13 ($AX = BX = CX = 000D$).

Vì hậu tố b / h / không hậu tố chỉ để báo cho assembler biết cách đọc con số lúc biên dịch.

Biên dịch xong, cả ba đều thành cùng một giá trị nhị phân giống hệt nhau trong mã máy: mã lệnh lần lượt là B8 0D 00, BB 0D 00, B9 0D 00, cùng chung byte 0D.

Lưu ý: máy luôn lưu trữ ở hệ nhị phân. Việc emulator hiển thị bằng hexa là chuyện riêng, chỉ để người đọc dễ nhìn.

9. Chuyển đổi qua lại giữa các hệ số (nêu cách tính toán, và tính bằng tay):

a. Chuyển số 1234 (Thập phân - Decimal) sang 2 hệ số còn lại.

- Dùng phương pháp tìm lũy thừa 2 lớn nhất rồi trừ dần (bài làm tay ở dưới):

$$1234 = 2^10 + 2^7 + 2^6 + 2^4 + 2^1 = 10011010010b$$

Sang hexa thì gom 4 bit từ phải sang trái: 100 1101 0010 $\rightarrow$ 4 D 2, vậy $1234 = 4D2h$.

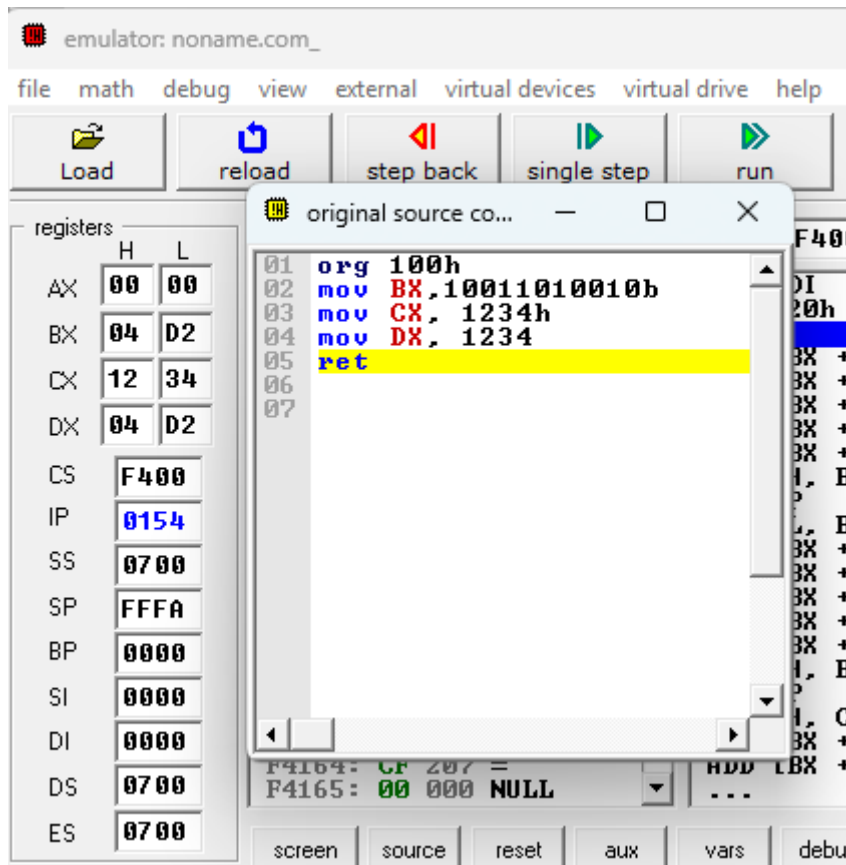
b. Chuyển số 11011011b (Nhị phân - Binary) sang 2 hệ số còn lại.

- Đánh số lần lượt các giá trị từ 0 $\rightarrow$ số cuối cùng theo thứ tự từ phải sang trái, rồi nhân lần lượt $a \cdot 2^n$ với a là giá trị nhị phân theo n, n là số thứ tự đã được đánh số và, thập lục phân thì dựa vào số thập phân đã được chuyển của 4 bit rồi dò.

c. Chuyển số 0F4h (Thập lục phân - Hexadecimal) sang 2 hệ số còn lại.

- Lần lượt chuyển hexadecimal về nhị phân rồi chuyển từ nhị phân sang decimal hoặc đổi theo dạng ghép A=10, B=11, ... F=15, rồi thế công thức $a \cdot 16^n$ cũng sẽ ra đáp án tương tự
- Bài làm các câu trên:

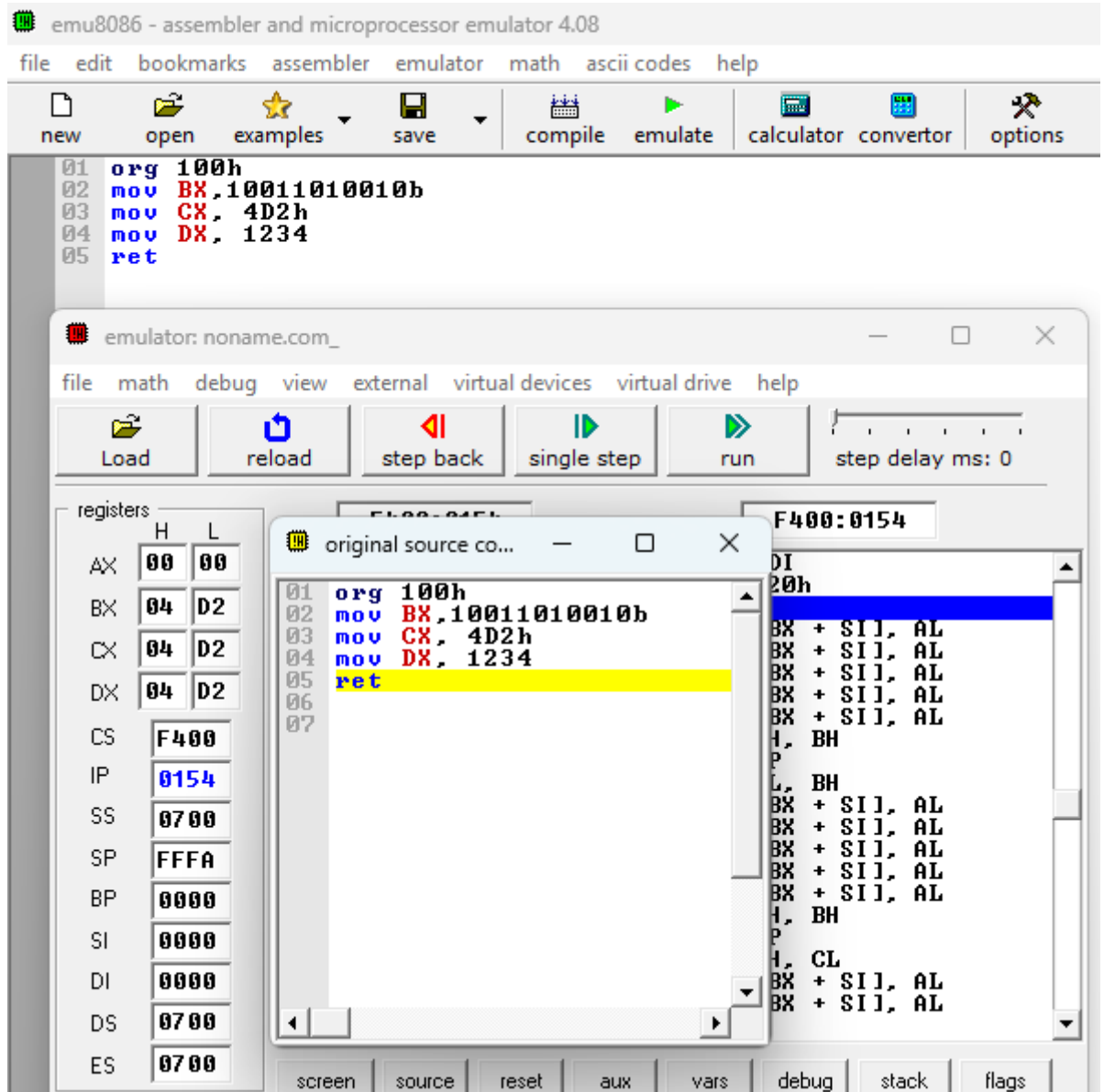
- Lỗi 1 - ở thanh ghi BX: viết 1234b là sai vì hệ nhị phân chỉ có chữ số 0 và 1, không tồn tại chữ số 2, 3, 4. Assembler báo "wrong parameters: MOV BX, 1234b". Sửa: viết đúng dạng nhị phân của 1234 là 10011010010b.
- Lỗi 2 - ở thanh ghi DL: DL là thanh ghi 8 bit, chỉ chứa được 0 đến 255, trong khi 1234 cần 11 bit. Assembler báo "operands do not match: second operand is over 8 bits". Sửa: đổi DL thành DX (16 bit).
- Lỗi 3 - ở thanh ghi CX, sai giá trị chứ không sai cú pháp: 1234h = 4660 chứ không phải 1234. Đề yêu cầu nạp giá trị 1234 ở dạng hexa nên phải viết 4D2h. Lỗi này assembler không báo vì cú pháp vẫn hợp lệ, chạy vẫn trơn tru, chỉ có kết quả sai. Ảnh dưới cho thấy BX và DX đều bằng 04D2 nhưng CX lại bằng 1234:



- o Sau khi sửa cả 3 lỗi trên, code cuối cùng là:
- o org 100h
- o mov BX, 10011010010b
- o mov CX, 4D2h
- o mov DX, 1234
- o ret

Chụp hình minh chứng.

- Kết quả cuối cùng:



Bài 4: Ký tự và bảng ASCII (MQH ký tự và số)

1. VDK, VXL có hiểu được ký tự ‘a’, ‘1’, ‘#’ không? (Gợi ý: Không, vì đây là cái ký tự trong ngôn ngữ của con người, máy móc không hiểu được). Vậy làm cách nào để Máy móc phân biệt được các ký tự này với nhau? (Trả lời câu số 2 là sẽ hiểu)
 - Không, vì đây là ký hiệu của con người, máy móc sẽ phân biệt các ký tự này dựa trên một bảng mã được quy định từ trước, gọi là bảng ASCII
2. Bản chất, khi các ký tự được lưu vào bộ nhớ, thì sẽ lưu dưới dạng gì?
 - Lưu dưới dạng số nhị phân, chính là mã ASCII của ký tự đó, mỗi ký tự chiếm 1 byte. Ví dụ khai báo x db ‘A’ thì trong bộ nhớ chỉ có duy nhất giá trị 41h, hoàn toàn không có khái niệm "chữ A" nào được lưu cả. Hệ hexa chỉ là cách hiển thị cho người đọc, trong chip không tồn tại hệ hexa.
3. Lập bảng, xác định giá trị số (2 dạng: Decimal, Hexa) tương ứng với các ký tự sau: ‘a’, ‘D’, ‘#’, ‘1’, ‘0’, ‘9’.

‘a’	097	61h
‘D’	068	44h
‘#’	035	23h
‘1’	049	31h

'0'	04h	30h
'9'	09h	39h

4. Chương trình ASM (8086) có lệnh giúp nhập 1 số “trực tiếp” từ bàn phím không? (Gợi ý: Không nha, chỉ có lệnh nhập vào 1 ký tự)
 - Không, vì nó chỉ cho phép nhập 1 ký tự.
5. Vậy, muốn nhập 1 ký tự từ bàn phím thì mình làm sao? (Gợi ý: Học sau, Dùng INT21h/AH=1, học trong video về “Nhập/Xuất ký tự từ bàn phím” nha.)
 - Dùng ngắt INT 21h với AH = 1. Sau khi thực hiện, mã ASCII của phím vừa bấm nằm trong AL, đồng thời ký tự đó được hiện luôn lên màn hình.
`mov ah, 1`
`int 21h` ; mã ASCII của phím -> AL
6. Chuyển ký tự số thành số thì làm sao?
 - Lấy mã ASCII của ký tự đó trừ đi 30h (tức trừ đi mã của ký tự '0').
 Ví dụ bấm phím '3' thì AL nhận mã ASCII là 33h chứ không phải số 3. Lấy 33h - 30h = 3, đó mới là con số dùng để tính toán được. Cách này áp dụng cho mọi ký tự số từ '0' đến '9'.
`sub al, 30h`
7. Muốn nhập 1 số có nhiều chữ số từ bàn phím thì phải làm sao? (Trả lời lý thuyết, từng bước làm là gì)
 - Không có cách nhập một lần, phải nhập từng ký tự rồi tự ghép lại thành số. Các bước:
 1. Đặt kết quả ban đầu bằng 0.
 2. Nhập 1 ký tự bằng INT 21h/AH=1.
 3. Nếu ký tự vừa nhập là Enter (mã 0Dh) thì dừng, kết quả đã xong.
 4. Ngược lại, đổi ký tự thành số bằng cách trừ 30h.
 5. Lấy kết quả = kết quả x 10 + chữ số vừa nhập.
 6. Quay lại bước 2.
 - Ví dụ nhập 1234: gõ '1' thì $0 \times 10 + 1 = 1$; gõ '2' thì $1 \times 10 + 2 = 12$; gõ '3' thì $12 \times 10 + 3 = 123$; gõ '4' thì $123 \times 10 + 4 = 1234$; gõ Enter thì dừng.
 Nhớ: nhập thì NHẬP 10, xuất thì CHIA 10. Lúc nhập trong máy chưa có sẵn số nào để mà chia, người dùng đang gõ vào từng ký tự một.
8. Chương trình ASM (8086) có lệnh giúp xuất 1 số (có 1 chữ số) “trực tiếp” ra màn hình không? (Gợi ý: Không nha, chỉ có lệnh xuất vào 1 ký tự)
 - Không có, chỉ có lệnh xuất 1 ký tự. Muốn xuất một con số thì phải tự đổi số đó thành ký tự rồi mới xuất được.
9. Vậy, muốn xuất 1 ký tự ra màn hình thì làm sao?
 - Dùng ngắt INT 21h với AH = 2, ký tự cần xuất đặt trong DL.
`mov ah, 2`
`mov dl, al`
`int 21h`
10. Chuyển 1 số (có 1 chữ số) thành ký tự số thì làm sao?
 - Làm ngược lại với câu 6: thay vì trừ 30h thì bây giờ cộng thêm 30h.
 Ví dụ số 7 cộng 30h thành 37h, chính là mã ASCII của ký tự '7', xuất ra màn hình sẽ thấy chữ số 7.
`add al, 30h`
11. Muốn xuất 1 số có nhiều chữ số ra màn hình thì phải làm sao? (Trả lời lý thuyết, từng bước làm là gì)
 - Phải tách số ra từng chữ số rồi xuất từng ký tự một. Các bước:
 1. Chia số cho 10, số dư chính là chữ số hàng đơn vị.
 2. Lấy thương chia tiếp cho 10 để ra chữ số hàng chục, cứ thế cho tới khi thương bằng 0.

3. Vì các chữ số lấy ra theo thứ tự ngược (hàng đơn vị ra trước), phải đẩy từng chữ số vào Stack, tách hết rồi mới POP ra in ngược lại. Nếu in ngay theo thứ tự vừa lấy thì số 42 sẽ hiện ra thành "24".

4. Mỗi chữ số cộng 30h để thành kí tự, rồi xuất bằng INT 21h/AH=2.

Đây chính là cách thủ tục print_num_uns trong thư viện emu8086.inc hoạt động, và cũng là lý do thủ tục đó dài tới 82 byte trong khi phép chia chỉ vài dòng.

Bài 5: Thanh ghi và bộ nhớ

- Trong môn học này, chúng ta quan tâm đến bao nhiêu thanh ghi? Liệt kê các thanh ghi và chức năng của nó?
 - 14 thanh ghi
 - 8 Trong số đó là các thanh ghi dùng cho mục đích chung(general purpose), có thể dùng để chứa các giá trị tùy ý, có thể dùng cho các trường hợp đặc biệt.
 - AX** – Accumulator, chứa kết quả phép toán.
 - BX** – Base, làm con trỏ địa chỉ [BX].
 - CX** – Counter, lệnh LOOP tự trừ.
 - DX** – Data, chứa nửa cao kết quả nhân/chia
 - 4 Thanh ghi phân đoạn (segment),
 - CS: trỏ đến địa chỉ chứa code
 - DS: trỏ đến địa chỉ chứa data
 - ES: Extra segment register.
 - SS: trỏ đến địa chỉ vùng nhớ Stack
 - Ngoài ra còn **IP** (chứa offset của lệnh sắp thực thi) và **Flags** (mỗi bit là một cờ trạng thái, cập nhật sau phép toán) → tổng cộng 14
- Thanh ghi nằm trong đâu?
 - Nằm trong CPU
 - Truy cập nhanh nhất hệ thống, vì không phải đi qua bus
 - Thanh ghi **không có địa chỉ bộ nhớ**, gọi bằng **tên**
 - Số lượng cực ít → phải liên tục luân chuyển dữ liệu giữa thanh ghi và bộ nhớ
- Kích thước của thanh ghi là bao nhiêu? (2 loại kích thước). Có thể sử dụng 1 thanh ghi để chứa 2 số khác nhau không?
 - Hai loại kích thước: **8 bit** (0–255) và **16 bit** (0–65535). Chỉ **AX, BX, CX, DX** tách được thành hai nửa AH/AL...; SP, BP, SI, DI không tách được. Chứa 2 số khác nhau thì được — AH một số, AL một số — nhưng đọc cả AX thì ra một số duy nhất theo $AX = AH \times 256 + AL$.
- Hỏi DX bằng nhiêu, sau khi đoạn chương trình sau? Org 100h MOV DL, 50 MOV DH, 220 RET
 - Giá trị DX sẽ là: **DC32h**
- Lệnh, biến được nạp và lưu vào đâu? (Gợi ý: Mỗi lệnh, mỗi biến sẽ được lưu vào 1 hoặc vài ô nhớ, tùy vào kích thước giải mã của lệnh, hay kích thước của biến)
 - Sẽ được lưu vào 1 hoặc vài ô nhớ, tùy vào giải mã của lệnh và kích thước của biến
 - Ví dụ:
 - Lệnh: RET 1 byte · MOV CL, 10h 2 byte · MOV BX, 14 3 byte
 - Biến: DB 1 byte · DW 2 byte · DD 4 byte
- Hệ thống dựa vào gì để truy cập vào 1 ô nhớ bất kỳ? (Gợi ý: Mỗi ô nhớ có 1 địa chỉ)
 - Dựa vào địa chỉ của ô nhớ đó
- Địa chỉ của ô nhớ có mấy dạng hiển thị? Cho địa chỉ ô nhớ là 07213h, đây là địa chỉ gì, và bạn hãy chuyển sang dạng hiển thị còn lại?
 - Có 2 dạng 1 dạng là physical đó chính là dạng địa chỉ đề đang cho, dạng khác là logical, dạng logical thì là 0700h:0213h
 - Công thức chuyển : Vật lý= segmentx16+offset

Bài 6: Địa chỉ logic và địa chỉ vật lý

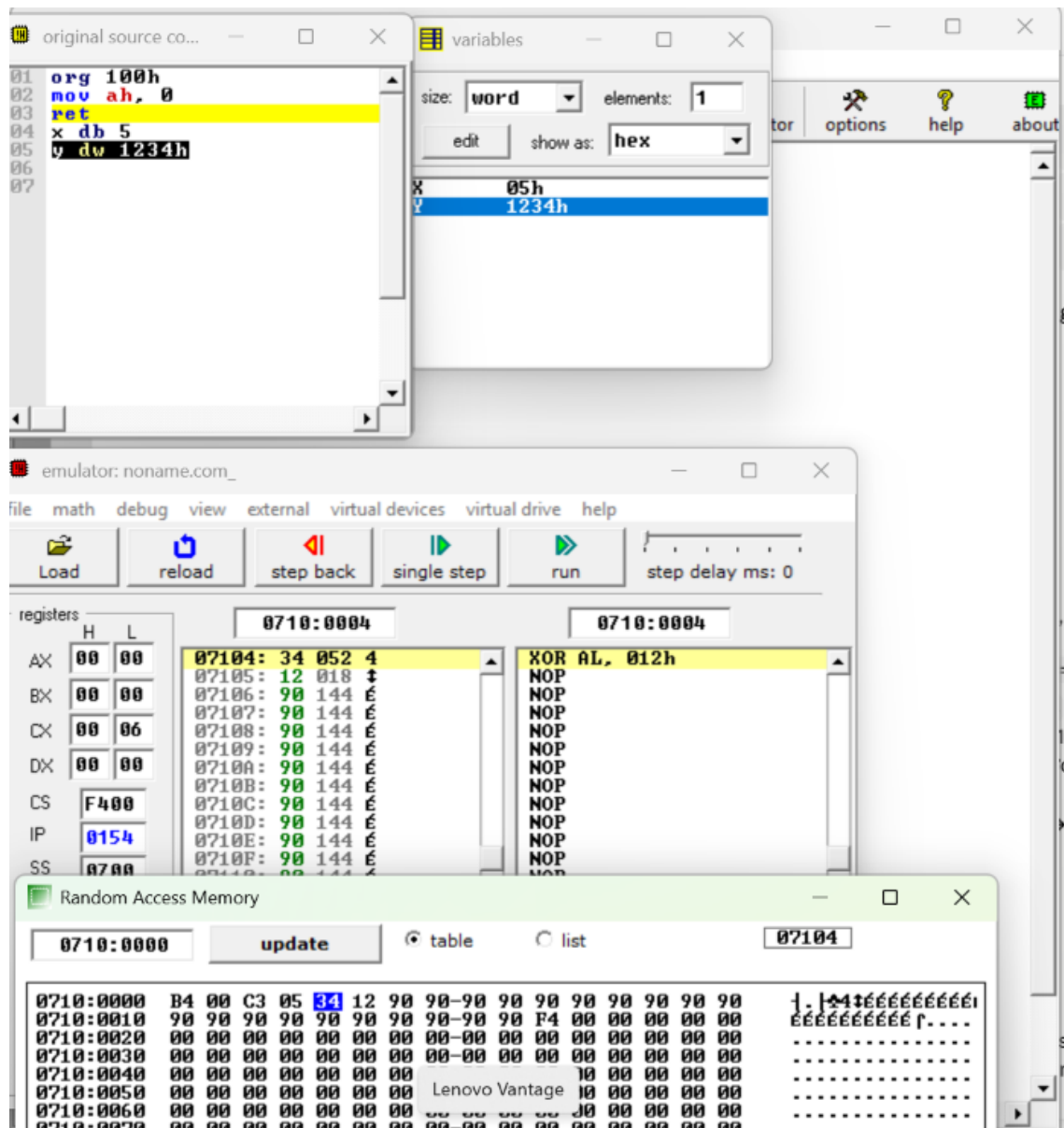
13. Chương trình, dữ liệu được lưu vào đâu?
- Được nạp vào **bộ nhớ RAM**, không nằm trong CPU. Vùng RAM dùng được cho chương trình và dữ liệu là từ **00500h đến A0000h**, rộng 654.080 byte. Dưới 00500h là bảng vector ngắt và vùng thông tin hệ thống, từ A0000h trở lên là bộ nhớ màn hình.
 - Trong emu8086, chương trình được nạp vào segment **0700h**, bắt đầu ở offset 0100h.
14. Địa chỉ của ô nhớ (1 byte) (trong bộ nhớ) có mấy dạng thể hiện?
- Có **2 dạng**:
 - o **Địa chỉ vật lý (physical)** — một số 20 bit, viết bằng 5 chữ số hexa, ví dụ 07105h. Đây là con số thật sự chạy trên 20 đường bus địa chỉ.
 - o **Địa chỉ logic (logical)** — viết dạng **segment:offset**, mỗi phần 16 bit, ví dụ 0700h:0105h. Đây là dạng mà chương trình và các thanh ghi làm việc.
 - Công thức chuyển: **Vật lý = Segment × 10h + Offset** (10h ở hệ 16 chính là 16 ở hệ 10). Viết "× 16h" là sai, vì 16h = 22.
 - Một địa chỉ vật lý ứng với **nhiều** cặp segment:offset khác nhau; ngược lại, một cặp segment:offset chỉ cho ra **một** địa chỉ vật lý duy nhất.
15. Cho địa chỉ 0700h:105h. Đây là địa chỉ gì? Segment là bao nhiêu? Offset là bao nhiêu? Địa chỉ vật lý tương ứng bằng bao nhiêu?
- Đây là **địa chỉ logic**, nhận ra nhờ dấu hai chấm ngăn giữa segment và offset.
 - Segment = **0700h**, Offset = **0105h**.
 - Vật lý = 0700h × 10h + 0105h = 07000h + 0105h = **07105h**.
16. Tất cả câu lệnh chương trình đều có chung địa chỉ Segment là bao nhiêu?
- Chung segment **0700h**, chính là giá trị nằm trong thanh ghi **CS**.
 - Cả chương trình nằm gọn trong một segment 64 KB nên CS **không đổi** trong suốt quá trình chạy; thứ thay đổi khi chạy từng lệnh là **IP**.
17. Địa chỉ segment của: vùng chứa chương trình, vùng chứa dữ liệu, vùng chứa Stack được lưu trong 3 thanh ghi nào? Có giá trị mặc định là bao nhiêu?
- Vùng chứa chương trình (mã lệnh) → **CS**; vùng chứa dữ liệu → **DS**; vùng chứa stack → **SS**.
 - Với chương trình COM trong emu8086, cả ba đều bằng **0700h** (thanh ghi ES cũng bằng 0700h). Ba vùng dùng **chung một segment 64 KB** — đây là đặc trưng của mô hình COM.
 - Giá trị segment này **cố định từ lúc chương trình được nạp**, không đổi khi chạy.
 - Xem được ở **bảng registers** trong cửa sổ emulator — như ảnh minh chứng.
18. Địa chỉ các câu lệnh chỉ khác nhau ở phần gì?
- Chỉ khác nhau ở phần **offset**. Phần segment giống hệt nhau vì cùng nằm trong một segment.
 - Offset của lệnh sau = offset của lệnh trước **cộng đúng số byte** mà lệnh trước chiếm.
19. Dòng lệnh đầu tiên của chương trình (dòng lệnh nằm sau ORG 100h) có địa chỉ logic, địa chỉ vật lý bằng bao nhiêu?
- Địa chỉ logic **0700h:0100h**, địa chỉ vật lý **07100h**.
 - Offset là 0100h chứ không phải 0 vì có **ORG 100h** — 256 byte đầu của segment được DOS dành cho **PSP**, chương trình không được đụng vào.
20. Trong video, lệnh MOV BL, 13 được lưu vào mấy ô nhớ (chiếm bao nhiêu byte)? Có địa chỉ bao nhiêu? Mã lệnh (Machine code, dạng hexa) là gì?
- Chiếm **2 byte** → nằm trong **2 ô nhớ**, vì mỗi ô nhớ chứa đúng 1 byte.
 - Địa chỉ: logic **0700h:0100h**, vật lý **07100h** (byte thứ hai nằm ở 07101h).
 - Mã máy: **B3 0D**.
 - o **B3** là opcode của "MOV BL, hằng số". Nhóm opcode **B0–B7** dành cho 8 thanh ghi 8 bit, BL đứng thứ 4 nên là B3.
 - o **0D** là số 13 đổi sang hexa. Hằng số chỉ tốn 1 byte vì đích BL là 8 bit.

21. Bạn viết lại chương trình trong video, và thêm 1 lệnh MOV BL, 14 vào sau lệnh trên, chạy kết quả và xem mã lệnh (Machine code) là gì?
- Mã máy: **B3 0E** — vẫn **2 byte**.
 - Opcode vẫn là **B3** vì đích vẫn là BL, chỉ có hằng số đổi từ 0Dh (13) sang **0Eh (14)**.
 - Lệnh này nằm ngay sau lệnh trước, ở offset **0102h** — logic 0700h:0102h, vật lý **07102h**. Đúng bằng 0100h cộng 2 byte của lệnh đứng trước.
22. Bạn thêm 1 lệnh MOV BX, 14 vào sau lệnh trên, chạy kết quả và xem mã lệnh (Machine code) là gì? Lệnh này chiếm bao nhiêu byte?
- Mã máy: **BB 0E 00** — chiếm **3 byte**, nằm ở offset **0104h** (vật lý **07104h**).
 - Khác với câu trên ở chỗ đích là **BX (16 bit)** chứ không phải BL (8 bit), và điều đó làm đổi hai thứ:
 - o Opcode nhảy từ nhóm B0–B7 (thanh ghi 8 bit) sang nhóm **B8–BF** (thanh ghi 16 bit); BX là **BB**.
 - o Hằng số phải chiếm đủ **2 byte** cho vừa thanh ghi 16 bit, nên 14 được ghi thành 0E 00 thay vì chỉ 0E.
 - Hai byte hằng số ghi theo kiểu **little-endian**: byte thấp (0E) đứng trước, byte cao (00) đứng sau.
 - So sánh gọn: **MOV BL, 14 = B3 0E (2 byte)** còn **MOV BX, 14 = BB 0E 00 (3 byte)**. Cùng một giá trị 14, khác nhau chỉ vì kích thước toán hạng đích.
 - **Đã chạy và xác nhận toàn bộ trên emulator** với chương trình gồm cả ba lệnh:

Lệnh	Địa chỉ (offset)	Mã máy	Số byte
`mov bl, 13`	`0100h`	`B3 0D`	2
`mov bl, 14`	`0102h`	`B3 0E`	2
`mov bx, 14`	`0104h`	`BB 0E 00`	3
`ret`	`0107h`	`C3`	1

- Cùng ảnh đó cũng cho thấy `CS = DS = SS = ES = 0700h` (câu 5) và `SP = FFFEh`, nên một tấm hình phức vụ được cả câu 5, 7, 8, 9 và 10.

- Lý do: **CPU không phân biệt được lệnh với dữ liệu** — trong bộ nhớ cả hai chỉ là byte như nhau. CPU cứ theo CS:IP mà đọc byte rồi giải mã thành lệnh, không hề biết byte đó là mã lệnh hay là số liệu.
 - Đặt sau ret thì khi chạy tới ret chương trình đã thoát, CPU không bao giờ chạm tới vùng dữ liệu đó nữa.
25. Có bao nhiêu cách xem kết quả của biến? Chụp hình minh chứng để xem biến y theo các cách bạn biết?
- Có 3 cách, mạnh dần:
 - o **Emulator / view / Variables** — hiện thẳng tên biến và giá trị. Phải chọn đúng kiểu hiển thị (byte hay word) thì mới đọc ra đúng số.
 - o **Emulator / view / Memory** — gõ địa chỉ dạng **0700:010A** rồi xem thẳng byte trong ô nhớ. Gõ thiếu phần segment sẽ không tìm ra ô nhớ.
 - o **Mở file .symbol** trong C:\emu8086\MyBuild — tra tên biến ra offset, rồi dùng offset đó cho cách 2.
 - Cần tới cách 3 vì sau khi biên dịch **tên biến biến mất** — assembler thay tên bằng địa chỉ số nhúng thẳng vào mã lệnh, trong bộ nhớ không còn chữ "y" nào cả.
 - Với biến `y dw 1234h` (2 byte), trong khung Memory sẽ thấy hai byte ghi theo **little-endian** — byte thấp trước, byte cao sau. Đo thực tế: `y` nằm ở offset **0104h**, ô `07104h` chứa **34h** và ô `07105h` chứa **12h**. Cửa sổ Variables thì gộp lại và hiện thẳng **1234h**.



Xem biến *y* bằng hai cách trong cùng một ảnh: cửa sổ **Variables** hiện `y = 1234h`, còn khung **Random Access Memory** cho thấy hai byte thật là `34` (ô 07104) rồi `12` (ô 07105) — đúng thứ tự *little-endian*. Đây cũng là phiên bản ĐÚNG của câu 4: biến nằm sau `ret`, và AX = 0000h.

26. Viết lại chương trình trong video, và di chuyển 2 lệnh khai báo biến *x*, *y* lên ngay phía sau ORG 100h. Vậy chuyện gì sẽ xảy ra?

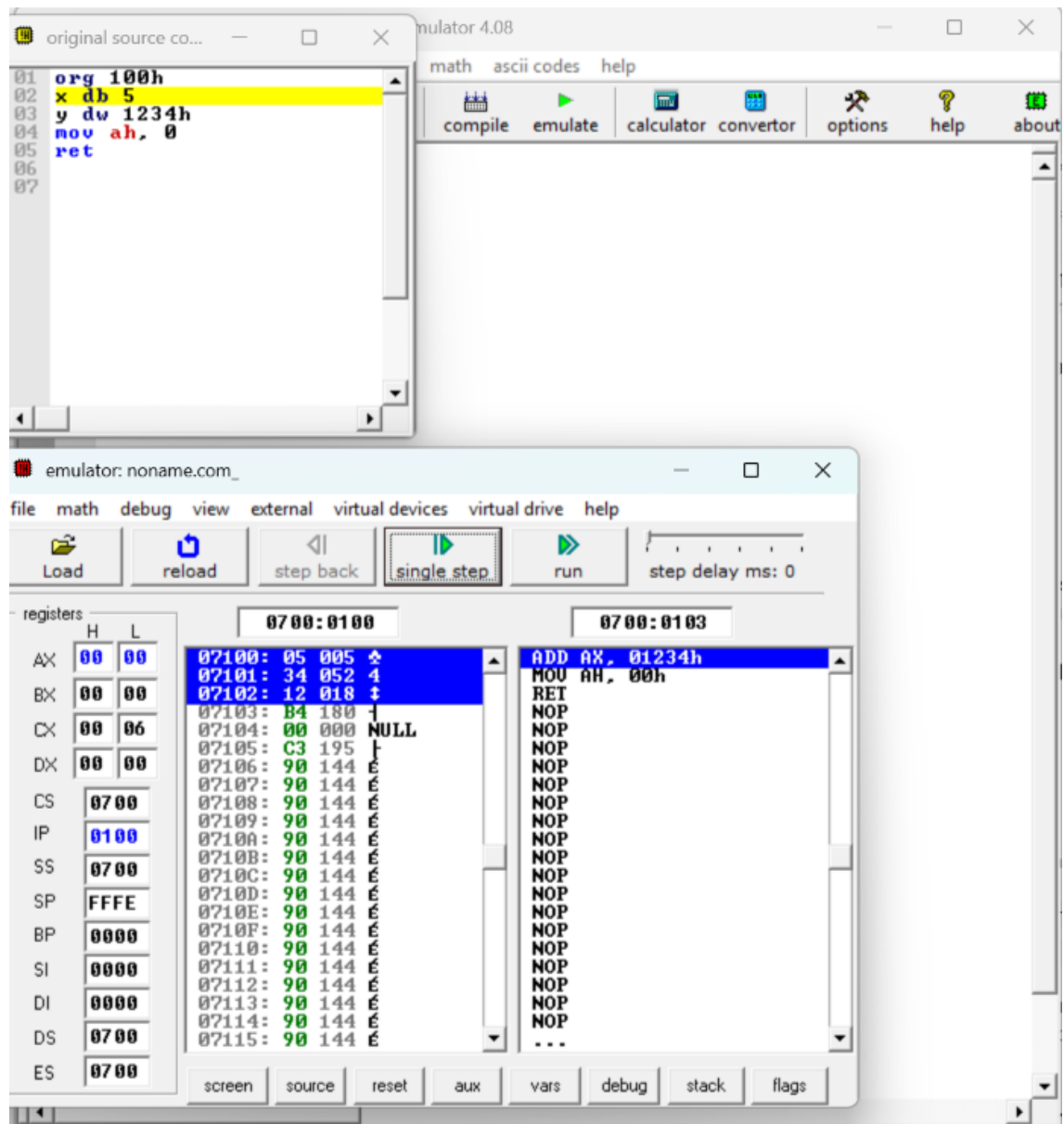
- Chương trình **chạy sai** — nhưng không phải sai kiểu treo máy, mà sai kiểu **CPU thực thi luôn dữ liệu như thể nó là mã lệnh**. Đây mới là điều đáng nói.
- **Phiên bản đúng** (biến đặt sau `ret`):

```
org 100h
mov ah, 0
ret
x db 5
y dw 1234h
```

- Byte trong bộ nhớ: `B4 00` (mov ah, 0) ở 07100, `C3` (ret) ở 07102, rồi mới tới dữ liệu — `05` của *x* ở 07103, `34 12` của *y* ở 07104 và 07105. CPU chạy từ 07100, gặp `ret` ở 07102 là thoát, **không bao giờ chạm tới 3 byte dữ liệu**. Chạy xong AX = 0000h.
- **Phiên bản sai** (dời biến lên ngay sau org 100h):

```
org 100h
```

- x db 5
- y dw 1234h
- mov ah, 0
- ret
- Byte trong bộ nhớ giờ là: `05` ở 07100, `34 12` ở 07101-07102, `B4 00` ở 07103-07104, `C3` ở 07105. **Giá trị các byte không hề đổi, chỉ đổi chỗ.**
- Nhưng CPU vẫn bắt đầu ở **CS:0100h**. Nó đọc `05 34 12` và bộ giải mã dịch đúng ba byte đó thành **một lệnh `ADD AX, 1234h`** — vì `05` chính là opcode của "ADD AX với hằng số 16 bit", và `34 12` được đọc là hằng số 1234h theo little-endian. Khung disassembly của emulator hiện thẳng dòng **`ADD AX, 01234h`** tại địa chỉ 07100.

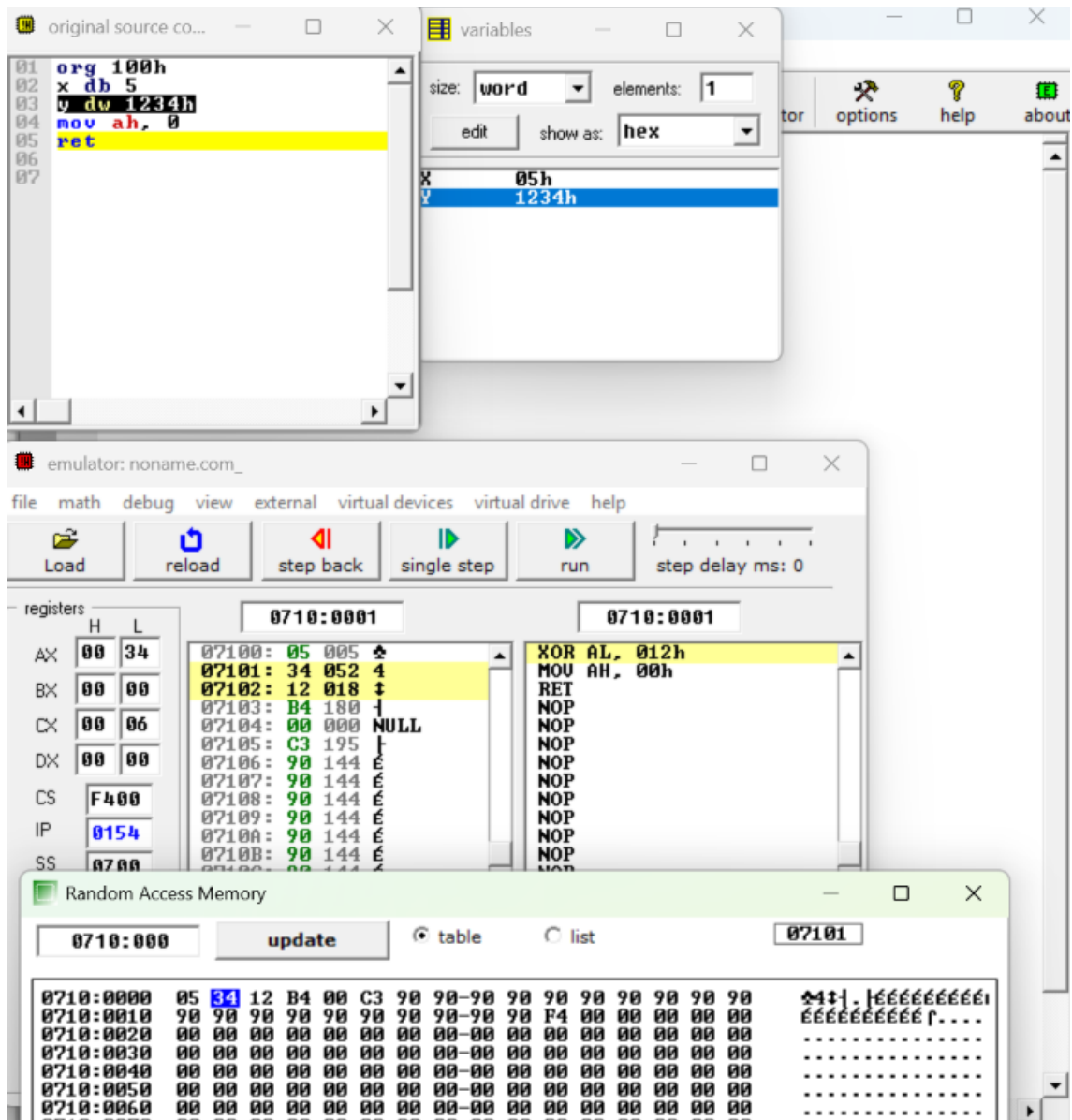


Phiên bản SAI, chụp ngay lúc IP = 0100h trước khi chạy. Khung disassembly hiện rõ **`ADD AX, 01234h`** tại địa chỉ 07100 — ba byte dữ liệu của hai biến x và y đã bị bộ giải mã ghép thành một lệnh, dù trong mã nguồn chúng chỉ là khai báo biến.

- Sau đó CPU chạy tiếp `B4 00` (`MOV AH, 0`) rồi `C3` (`RET`) và thoát.
- **Bằng chứng rõ nhất nằm ở thanh ghi AX:**

Phiên bản	AX sau khi chạy	Vì sao
-----------	-----------------	--------

Biến đặt sau `ret` (đúng)	0000h	CPU không đụng tới vùng dữ liệu
Biến đặt trước lệnh (sai)	0034h	`ADD AX, 1234h` làm AX = 1234h, rồi `MOV AH, 0` xoá nửa cao, còn lại AL = 34h

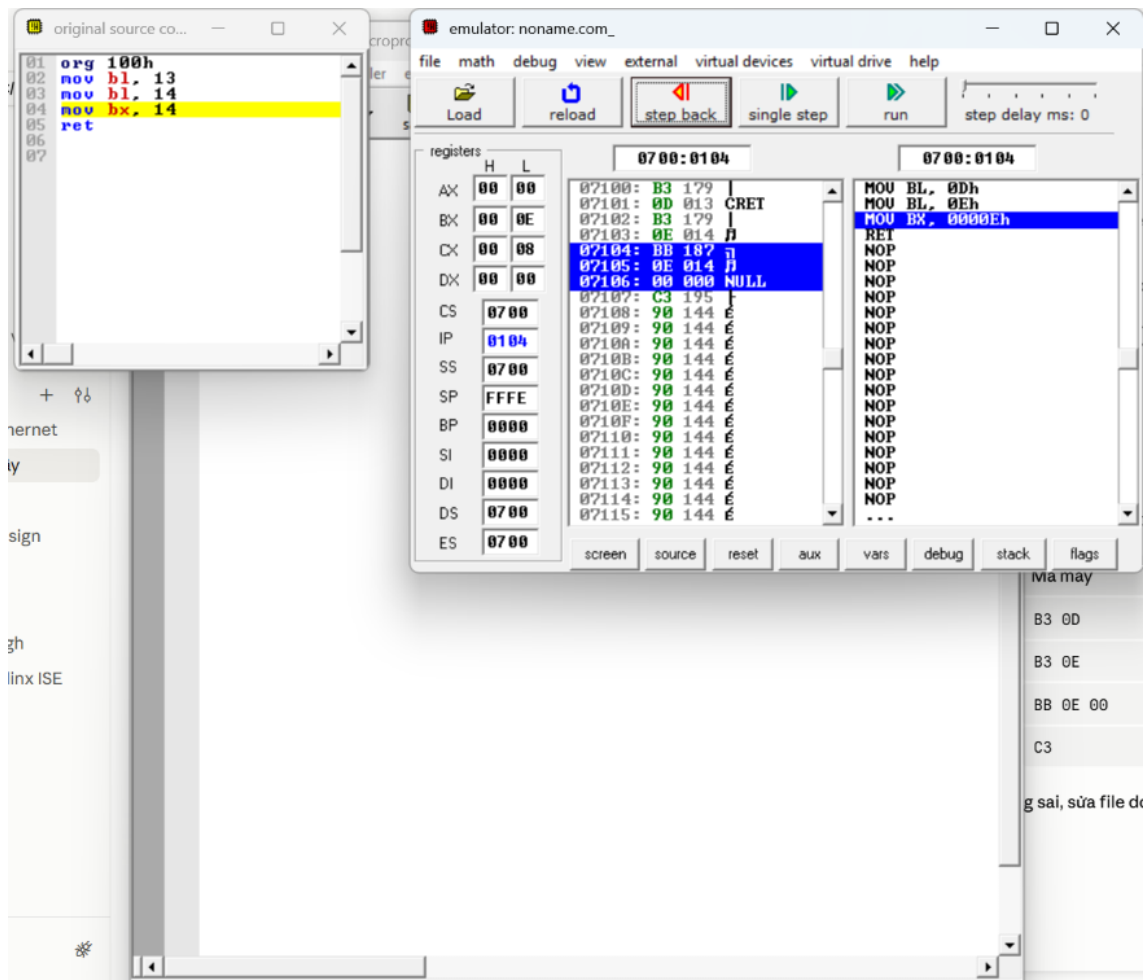


Phiên bản SAI, sau khi chạy xong. Thanh ghi **AX = 0034h** thay vì 0000h — đúng bằng byte thấp của biến y bị lệnh ma `ADD AX, 1234h` cuốn vào. Khung bộ nhớ cho thấy dãy byte `05 34 12 B4 00 C3`, và cửa sổ Variables vẫn đọc ra x = 05h, y = 1234h hoàn toàn bình thường — dữ liệu không hỏng, chỉ là nó đã bị chạy qua.

- **⚠ Chỗ tinh tế và gần như chắc chắn bị hỏi vặn:** ở ví dụ này chương trình **không hề treo**. Ba byte dữ liệu tình cờ ghép thành đúng **một lệnh 3 byte hợp lệ**, nên chạy xong nó rơi đúng vào `B4 00` và **tự đồng bộ lại được**.
- Đó là **may mắn, không phải quy luật**. Nếu dữ liệu ghép thành lệnh dài 2 byte hoặc 4 byte thì mọi byte phía sau bị lệch: `MOV AH, 0` và `RET` sẽ bị giải mã thành lệnh hoàn toàn khác, và lúc đó chương trình mới thật sự chạy loạn hoặc treo.
- Kết luận đúng bản chất: thứ quyết định một byte là **lệnh** hay **dữ liệu** không phải bản thân byte đó, mà là **CPU có chạy tới nó hay không**. Dãy `05 34 12` vừa có thể là "số 5 và số

- 1234h", vừa có thể là "lệnh ADD AX, 1234h" — hoàn toàn tùy vào việc IP có trỏ vào nó hay không.
- Cùng bản chất đó: trong khung disassembly, phần nằm sau ``ret`` hiện một dãy dài ``NOP``. Đó không phải chương trình — đó là byte ``90h`` trong bộ nhớ trống bị bộ giải mã dịch thành lệnh, vì ``90h`` chính là mã máy của ``NOP``.
27. Thanh ghi IP chứa gì? Chức năng của thanh ghi IP là gì?
- IP (Instruction Pointer) chứa **offset của lệnh sắp được thực thi**, tính trong segment mã lệnh.
 - Ghép với CS thành **CS:IP** — địa chỉ logic đầy đủ của lệnh kế tiếp. IP chỉ giữ phần offset, phần segment do CS giữ.
 - Chức năng: cho CPU biết phải nạp lệnh nào tiếp theo. Sau mỗi lần nạp lệnh, IP **tự động cộng thêm đúng số byte của lệnh vừa nạp**, nhờ vậy nó tự trỏ sang lệnh liền sau.
 - Các lệnh nhảy (JMP, CALL, LOOP, các lệnh J điều kiện) làm việc bằng cách **ghi giá trị mới vào IP** — đó là toàn bộ cơ chế rẽ nhánh.
 - IP là thanh ghi **16 bit, không tách đôi được**, và **không ghi trực tiếp bằng MOV** — không tồn tại lệnh MOV IP, ...
28. Trong video, khi lệnh MOV DH, 5Dh vừa thực hiện, thì dòng lệnh nào sẽ được highlight vàng, giá trị trong thanh ghi IP bằng bao nhiêu, lệnh chuẩn bị được nạp vào CPU để giải mã và thực hiện là lệnh nào?
- Lệnh MOV DH, 5Dh có mã máy **B6 5D**, chiếm **2 byte** (B6 là opcode MOV DH với hằng số, 5D là hằng số).
 - Nguyên tắc: khi một lệnh **vừa thực hiện xong**, IP đã được cộng thêm số byte của chính lệnh đó, nên IP **không còn trỏ vào lệnh vừa chạy mà trỏ sang lệnh liền sau**.
 - Vậy: **IP = offset của lệnh MOV DH, 5Dh + 2**, và lệnh chuẩn bị được nạp vào CPU để giải mã là **lệnh nằm ngay sau MOV DH, 5Dh**, tức lệnh tại địa chỉ CS:IP.
 - Dòng được highlight chính là dòng **tại CS:IP** — dòng sắp chạy, không phải dòng vừa chạy xong.
 - **Đã xác nhận trực tiếp trên emulator**. Chạy chương trình ``mov bl, 13` / `mov bl, 14` / `mov bx, 14` / `ret`, bấm single step cho tới khi hai lệnh đầu chạy xong:

 - o `IP = 0104` — đúng bằng 0100h + 2 byte + 2 byte của hai lệnh đã chạy.
 - o Dòng được tô vàng là dòng `mov bx, 14` — tức dòng nằm tại CS:IP, dòng chưa chạy, không phải dòng vừa chạy xong.
 - o Khung disassembly cũng tô sáng đúng lệnh đó (`MOV BX, 0000Eh` tại địa chỉ 07104), khớp hoàn toàn với dòng vàng bên cửa sổ source.`
 - Vậy quy tắc là: **dòng vàng luôn đi theo IP**, và IP luôn trỏ tới lệnh **sắp** thực thi.



Bằng chứng cho quy tắc trên: IP = 0104h, và dòng tô vàng bên cửa sổ source đang nằm ở `mov bx, 14` — chính là lệnh tại địa chỉ 07104 mà khung disassembly bên phải cũng tô sáng. Hai lệnh phía trên đã chạy xong (BL = 0Eh) nhưng không còn được tô.

Bài 8: [Optional] Cấu trúc chương trình với các vùng tách riêng

Bài này thầy không đặt câu hỏi (phần mô tả để trống). Đây là **mô hình EXE**, khác với mô hình COM đang dùng ở chỗ mã lệnh, dữ liệu và stack được khai báo thành **các segment tách riêng**.

```
.model small
.stack 100h
.data
    x db 5
.code
main proc
    mov ax, @data
    mov ds, ax        ; phải tự nạp DS, DOS không đặt sẵn
    ; ... phần thân
    mov ah, 4Ch
    int 21h           ; kết thúc chương trình kiểu EXE
main endp
end main
```

- Khác biệt giữa hai mô hình:

	COM (đang dùng)	EXE
Segment	CS = DS = SS = ES, chung 64 KB	Tách riêng, CS ≠ DS ≠ SS

Điểm bắt đầu	`org 100h`, offset 0100h (256 byte đầu là PSP)	Không cần org 100h
Thanh ghi DS	DOS đặt sẵn	Phải tự nạp: mov ax, @data / mov ds, ax
Kích thước tối đa	64 KB	Không bị giới hạn 64 KB
Cách kết thúc	`ret`	`mov ah, 4Ch` rồi `int 21h`

- Lý do phải nạp DS bằng tay ở EXE: lúc DOS nạp chương trình, nó chỉ đặt CS và SS đúng chỗ, còn DS và ES thì trở vào **PSP** chứ không trở vào vùng dữ liệu của mình. Không nạp lại DS thì mọi lệnh đọc biến sẽ đọc nhầm vùng.

Bài 9: Quá trình biên dịch, mã hóa, bộ nhớ

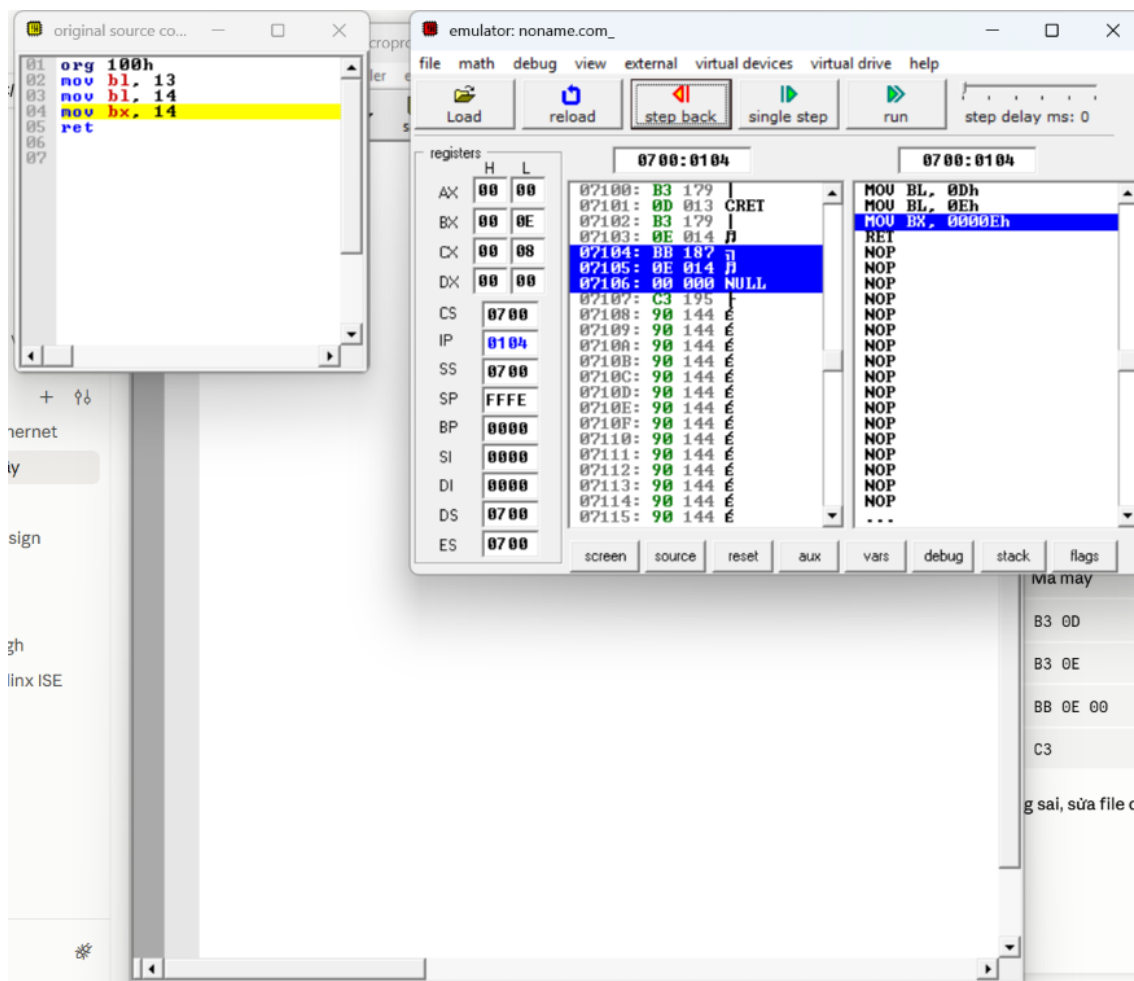
Thầy ghi rõ "Không có câu hỏi nha". Nhưng nội dung bài này là nền cho rất nhiều câu hỏi vắn ở các bài khác, nên tóm tắt lại:

- Đường đi của một chương trình: **Mã nguồn (.asm) → Assembler → mã máy (.com) → nạp vào bộ nhớ → CPU thực thi.**
- Assembler làm 3 việc, và làm **xong hết ở bước biên dịch**, tức là trước khi chương trình chạy:
 - o **Quy đổi mọi hệ đếm về nhị phân.** `MOV AX, 1101b`, `MOV BX, 0Dh`, `MOV CX, 13` cho ra mã máy `B8 0D 00`, `BB 0D 00`, `B9 0D 00` — cùng byte `0D`.
 - o **Emu8086 không chuyển đổi hệ đếm lúc chạy.**
 - o **Thay tên biến, tên nhãn bằng địa chỉ số.** Sau khi dịch, tên biến biến mất hoàn toàn khỏi mã máy; trong bộ nhớ chỉ còn byte giá trị và địa chỉ.
 - o **Chọn opcode và tính độ dài lệnh** theo kích thước toán hạng. Vì vậy `MOV CL, 10h` ra 2 byte còn `MOV CX, 10h` ra 3 byte.
- Emu8086 sinh 4 file trong `C:\emu8086\MyBuild` sau khi bấm compile:
 - o **`.com`** — mã máy, chính là chương trình chạy được
 - o **`.list`** — bảng đối chiếu **địa chỉ ↔ mã máy ↔ dòng lệnh nguồn**. Đây là bằng chứng mạnh nhất khi cần chứng minh một lệnh chiếm mấy byte
 - o **`.symbol`** — bảng tra **tên biến → offset**, dùng khi cần biết biến nằm ở ô nhớ nào
 - o **`.debug`** — dữ liệu phục vụ trình gỡ lỗi

Bài 10: Biến đơn và bộ nhớ dữ liệu

- Giá trị của biến được lưu trong vùng bộ nhớ chương trình hay bộ nhớ dữ liệu?
 - Lưu trong **vùng bộ nhớ dữ liệu**, do thanh ghi **DS** quy định.
 - Nhưng với chương trình COM thì hai vùng đó **nằm chung một segment 64 KB** (CS = DS = 0700h), nên trên thực tế biến nằm ngay sau phần mã lệnh, trong cùng một segment. Hai vùng chỉ khác nhau về **vai trò**, không khác nhau về segment.
- Chương trình có hiểu được tên biến không? Tên biến có lưu trong bộ nhớ không?
 - **Không, và không.**
 - Tên biến chỉ tồn tại trong **mã nguồn**, phục vụ người viết và assembler. Khi biên dịch, assembler **thay tên bằng địa chỉ số** nhúng thẳng vào mã lệnh.
 - Lúc chương trình chạy, trong bộ nhớ **không còn chữ "var1" nào cả** — chỉ có byte giá trị và địa chỉ của nó. CPU chỉ biết địa chỉ, không biết tên.
 - Muốn tra ngược tên thành địa chỉ thì mở file **`.symbol`** trong `C:\emu8086\MyBuild`. File này do assembler sinh ra để phục vụ debug, **không nằm trong file chạy.**
- Địa chỉ segment của vùng bộ nhớ dữ liệu là bao nhiêu? Được lưu trong thanh ghi gì?
 - Bằng **0700h**, lưu trong thanh ghi **DS** (Data Segment).

32. Theo chương trình trong video, câu lệnh và biến đều có địa chỉ segment là như nhau. Đúng hay sai? Giải thích và đưa hình ảnh để minh chứng?
- **Đúng.** Trong chương trình COM, CS = DS = SS = ES = **0700h**.
 - Giải thích: file COM không khai báo segment riêng. DOS nạp toàn bộ chương trình vào **một segment duy nhất** rồi đặt cả 4 thanh ghi đoạn về cùng giá trị đó. Mã lệnh, dữ liệu và stack vì vậy dùng chung một segment 64 KB, chỉ khác nhau ở phần **offset**.
 - Điều này **không đúng với mô hình EXE** — EXE khai báo .code, .data, .stack tách riêng nên CS khác DS khác SS (xem Bài 8).
 - Minh chứng: bảng **registers** trong cửa sổ emulator cho thấy cả 4 thanh ghi đoạn cùng bằng 0700h — như ảnh minh chứng.



Bảng **registers** lúc chương trình COM đang chạy: **CS = DS = SS = ES = 0700h**. Bốn thanh ghi đoạn bằng nhau nên mã lệnh và biến dùng chung một segment, đúng như câu trả lời. Giá trị này giống nhau ở mọi chương trình COM, vì DOS đặt cả bốn thanh ghi đoạn về cùng một segment ngay lúc nạp chương trình.

33. Có mấy cách để khai báo biến x có giá trị là 123? Viết ra câu lệnh đó?

- Cùng một giá trị 123, viết được ở nhiều hệ đếm và nhiều kích thước:
 - x db 123 ; hệ 10, 1 byte
 - x db 7Bh ; hệ 16, 1 byte
 - x db 01111011b ; hệ 2, 1 byte
 - x dw 123 ; 2 byte, nếu cần biến 16 bit
- Ba cách đầu assembler đều dịch ra **cùng một byte 7Bh**; khác nhau chỉ ở **cách viết cho người đọc**, không khác gì trong bộ nhớ.
- Cách thứ tư khác thật sự: ``dw`` cấp phát **2 byte** (7B 00 theo little-endian) chứ không phải 1.
- Đây lại là ý cốt lõi của Bài 3: máy chỉ lưu **nhị phân**, hệ đếm chỉ là chuyện của người viết và được assembler quy đổi lúc biên dịch.

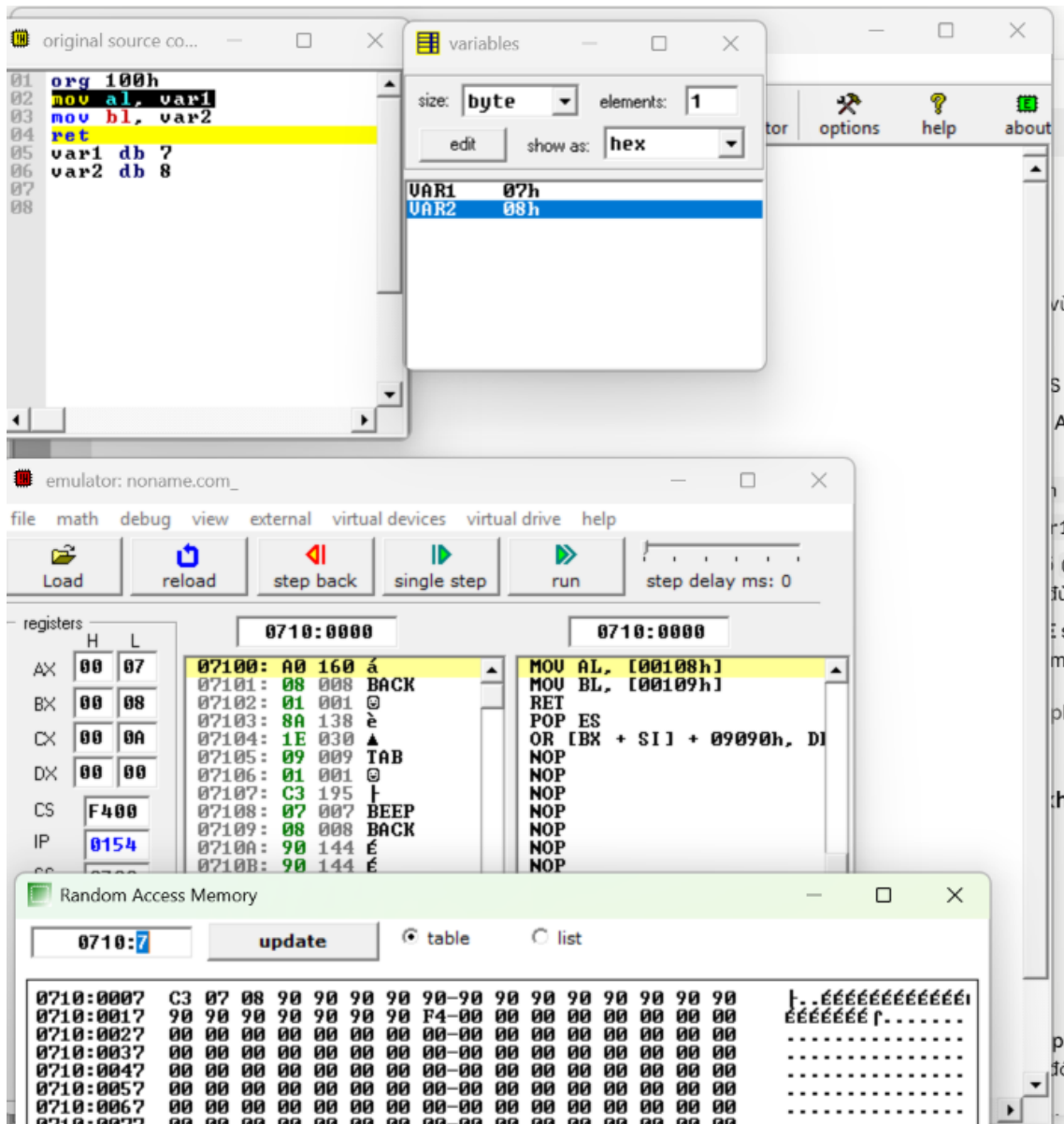
34. Theo video, lệnh MOV AX, var1 bị báo lỗi, lỗi này là gì?
- Lỗi **không khớp kích thước toán hạng** — emu8086 báo "operand types do not match".
 - Lý do: `var1` khai báo bằng **`db`** nên chỉ là **1 byte (8 bit)**, còn **AX là thanh ghi 16 bit**. Assembler không tự quyết được phải lấy thêm byte nào cho đủ 16 bit, nên báo lỗi ngay lúc biên dịch chứ không đợi tới lúc chạy.
 - Cách sửa: dùng thanh ghi 8 bit, tức **`MOV AL, var1`**. Hoặc nếu thật sự cần 16 bit thì phải khai báo lại biến là **`var1 dw ...`**.
 - Đây cùng một loại lỗi với **`MOV DL, 1234`** ở Bài 3 và **`MOV DH, 300`** ở Bài 5 — đều là lệch kích thước giữa hai toán hạng, chỉ khác chiều.
35. Vị trí khai báo biến nên để ở đâu?
- Đặt **sau lệnh `ret`**.
 - Lý do: **CPU không phân biệt được lệnh với dữ liệu** — trong bộ nhớ cả hai chỉ là byte như nhau. CPU cứ theo CS:IP đọc byte rồi giải mã thành lệnh. Nếu biến nằm trước, CPU sẽ chạy vào vùng dữ liệu và giải mã byte dữ liệu như mã lệnh, làm lệch toàn bộ luồng lệnh.
 - Đặt sau **`ret`** thì khi chạy tới **`ret`** chương trình đã thoát, không bao giờ chạm tới vùng đó nữa.
36. Biến var1, var2 chiếm bao nhiêu byte?
- Số byte do **từ khoá khai báo** quyết định, không phải do giá trị:

Khai báo	Tên đầy đủ	Số byte
<code>`db`</code>	Define Byte	1 byte
<code>`dw`</code>	Define Word	2 byte
<code>`dd`</code>	Define Doubleword	4 byte

- Trong chương trình của video, cả **`var1`** và **`var2`** đều khai báo bằng **`db`** nên mỗi biến chiếm **1 byte**. Biết chắc là **`db`** nhờ câu 6: **`MOV AX, var1`** báo lỗi lệch kích thước với thanh ghi 16 bit.
 - Kiểm chứng: mở **`Emulator / view / Memory`** hoặc file **`.symbol`** — địa chỉ của **`var2`** hơn địa chỉ của **`var1`** đúng **1**.
37. Mã lệnh (machine code) của lệnh RET là gì? Chiếm bao nhiêu byte?
- Mã máy là **`C3`**, chiếm đúng **1 byte**.
 - Đây là lệnh ngắn nhất trong nhóm đang học — không có toán hạng nên chỉ cần opcode, không cần byte nào đi kèm.
38. Địa chỉ logic, địa chỉ vật lý của biến var1 là bao nhiêu?
- Cách xác định, áp dụng được cho mọi chương trình chứ không cần thuộc lòng con số:
 - o Biến nằm **ngay sau lệnh `ret`**, nên offset của **`var1`** = 0100h + tổng số byte của tất cả các lệnh phía trước + 1 byte của **`ret`**.
 - o Địa chỉ **logic** = **`0700h : <offset đó>`**
 - o Địa chỉ **vật lý** = 0700h × 10h + offset = **07000h + offset**
 - **Kết quả đo được trên emulator** với chương trình của video:


```
org 100h
mov al, var1      ; A0 08 01    -> 3 byte, tại 07100
mov bl, var2      ; 8A 1E 09 01 -> 4 byte, tại 07103
ret               ; C3          -> 1 byte, tại 07107
var1 db 7         ; 07h        -> tại 07108
var2 db 8         ; 08h        -> tại 07109
```
 - Vậy **`var1`** nằm ở offset **0108h**:
 - o Địa chỉ **logic**: **0700h:0108h**
 - o Địa chỉ **vật lý**: **07108h**

- Kiểm chứng chắc chắn nhất: khung disassembly hiện lệnh đầu là `MOV AL, [00108h]` — assembler đã thay tên `var1` bằng đúng con số 0108h, đúng như câu 2 đã nói (tên biến biến mất sau khi biên dịch).



Ảnh dùng chung cho câu 8, 10, 11 và 15. Khung disassembly hiện `MOV AL, [00108h]` và `MOV BL, [00109h]` — hai địa chỉ hơn kém nhau đúng 1, xác nhận mỗi biến `db` chiếm 1 byte. Cửa sổ Variables cho `VAR1 = 07h`, `VAR2 = 08h`. Khung Random Access Memory mở tại `0710:0007` cho thấy byte `C3` (`ret`) rồi tới `07` và `08` là hai biến.

39. Địa chỉ logic, địa chỉ vật lý của biến `var2` là bao nhiêu?

- `var2` nằm **ngay sau** `var1`, mà `var1` là `db` chiếm 1 byte, nên **offset của var2 = offset của var1 + 1 = 0109h**.
 - o Địa chỉ **logic**: **0700h:0109h**
 - o Địa chỉ **vật lý**: **07109h**
- Khung disassembly hiện lệnh thứ hai là `MOV BL, [00109h]` — khớp đúng.
- **Một chi tiết rất dễ bị hỏi vắn nằm ngay ở hai lệnh này**: cùng là " nạp một byte từ ô nhớ vào thanh ghi 8 bit", nhưng

Lệnh	Mã máy	Số byte
<code>MOV AL, var1</code>	<code>A0 08 01</code>	3 byte

<code>`MOV BL, var2`</code>	<code>`8A 1E 09 01`</code>	4 byte
-----------------------------	----------------------------	---------------

- Lý do: **AL có một dạng mã lệnh rút gọn riêng**. Opcode ``A0`` nghĩa là " nạp vào AL từ địa chỉ trực tiếp", địa chỉ đi kèm 2 byte là xong. Còn các thanh ghi khác phải dùng dạng tổng quát ``8A`` cộng thêm **một byte ModRM** (``1E``) để chỉ rõ thanh ghi đích là BL, rồi mới tới 2 byte địa chỉ.
- Đây là ví dụ nữa cho ý ở Bài 5 và Bài 6: **độ dài lệnh phụ thuộc vào thanh ghi được gọi tên trong lệnh**, không phải vào giá trị dữ liệu.

40. Viết 1 câu lệnh Copy dữ liệu từ 1 ô nhớ (thông qua địa chỉ của ô nhớ) vào 1 thanh ghi?

```
mov al, [bx]
```

- Ý nghĩa: lấy **giá trị chứa trong ô nhớ** có offset đang nằm trong BX, segment mặc định là DS, rồi đưa vào AL.
- Dấu ngoặc vuông là mấu chốt: ``MOV AL, BL`` là copy **nội dung thanh ghi**, còn ``MOV AL, [BX]`` là copy **nội dung ô nhớ mà BX đang trỏ tới**. Bỏ ngoặc là đổi hẳn ý nghĩa.
- Chỉ **4 thanh ghi** được dùng làm con trỏ bộ nhớ: **[BX], [SI], [DI], [BP]**. Viết ``MOV AL, [AX]`` hay ``MOV AL, [CX]`` là **sai**, assembler báo lỗi.
- **⚠️ [BP] mặc định lấy segment `SS`**, còn `[BX]`, `[SI]`, `[DI]` mặc định lấy `DS`. Đây là ngoại lệ duy nhất và rất hay bị hỏi vặn.
- Muốn ghi ngược lại từ thanh ghi vào ô nhớ thì đảo hai toán hạng: ``mov [bx], al``.

41. Phương thức lưu số lớn (lớn hơn 1 byte) vào bộ nhớ là Big Endian hay Little Endian?

- **Little Endian.**
- Nghĩa là **byte thấp (LSB) lưu ở địa chỉ thấp**, byte cao lưu ở địa chỉ cao — ngược với thứ tự khi ta viết số ra giấy.
- Ví dụ đã gặp: ở Bài 5, địa chỉ ``010Ah`` được ghi vào bộ nhớ thành ``0A 01``; ở Bài 6, ``MOV BX, 14`` có mã máy ``BB 0E 00`` — hằng số `000Eh` ghi thành ``0E 00``.

42. Cho số `1122h` lưu vào ô nhớ bắt đầu địa chỉ là `07120h`, thì số này sẽ lưu vào ô nhớ nào với giá trị là bao nhiêu?

- Số `1122h` là số 16 bit nên chiếm **2 ô nhớ**, và ghi theo **little-endian**:

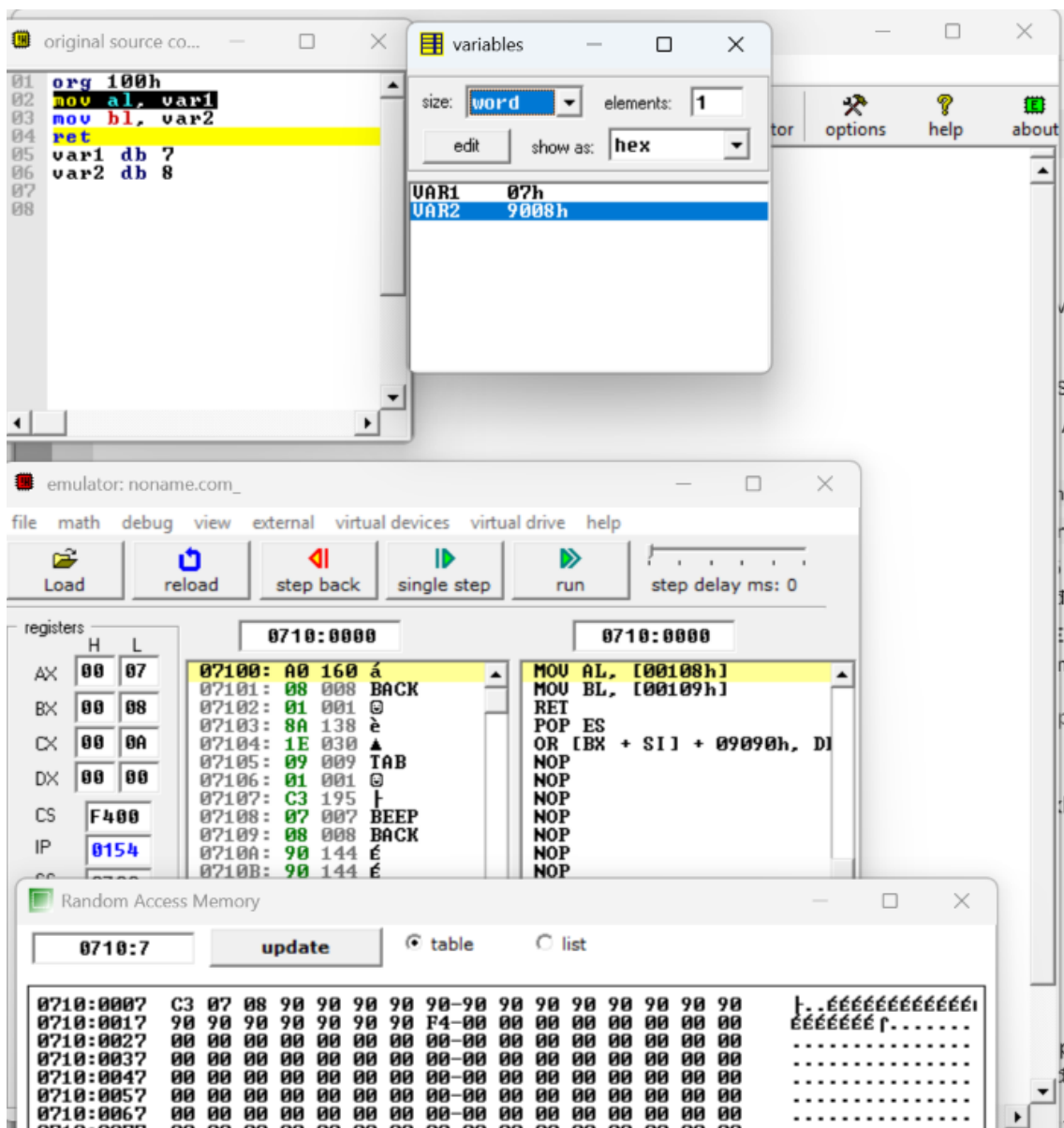
Địa chỉ ô nhớ	Giá trị	Vai trò
<code>`07120h`</code>	22h	byte thấp (LSB), nằm ở địa chỉ thấp
<code>`07121h`</code>	11h	byte cao (MSB), nằm ở địa chỉ cao

- Nếu đọc bằng mắt trong khung Memory sẽ thấy dãy ``22 11``, dễ tưởng nhầm là số `2211h`. Phải nhớ đảo lại mới ra giá trị thật **1122h**.

43. Lúc chạy chương trình, có những cách nào để xem giá trị của biến trong bộ nhớ?

- Có **3 cách**:
 - o **Emulator** (màn hình chính) — nhìn thẳng vào khung memory dump đang hiển thị, đọc byte tại địa chỉ của biến.
 - o **Emulator / view / Memory** — gõ địa chỉ dạng ``0700:010A`` rồi xem thẳng byte trong ô nhớ. Gõ thiếu phần segment sẽ không tìm ra ô nhớ.
 - o **Emulator / view / variables** — hiện thẳng tên biến và giá trị, dễ đọc nhất nhưng phải chọn đúng kiểu hiển thị.
- Cách 1 và 2 cho thấy **byte thật trong bộ nhớ**; cách 3 là bảng do emulator dựng lại từ bảng symbol, tiện hơn nhưng đã qua một lớp diễn giải.
- Chụp hình theo cả 3 cách, mỗi cách chụp đúng chỗ đang hiển thị giá trị của biến.

- Ảnh minh chứng cho cả ba cách chính là ảnh ở câu 10 phía trên — trong một tấm có đủ: khung memory dump ở màn hình chính emulator (cách 1), cửa sổ **Random Access Memory** mở tại `0710:0007` (cách 2), và cửa sổ **Variables** (cách 3).
44. Giải thích các thông số trong cửa sổ "Emulator/view/variables"?
- Cửa sổ này liệt kê các biến đã khai báo, mỗi dòng gồm:
 - o **Name** — tên biến trong mã nguồn. Lấy từ **bảng symbol** do assembler sinh ra, **không phải** đọc được từ bộ nhớ, vì trong bộ nhớ không hề có tên biến.
 - o **Address** — địa chỉ của biến, hiện dạng segment:offset hoặc chỉ offset. Đây là chỗ tra nhanh số cần cho câu 10 và 11.
 - o **Type / Size** — kiểu hiển thị: BYTE, WORD, DWORD. Đây chỉ là **cách đọc byte**, không phải thuộc tính của bộ nhớ.
 - o **Value** — giá trị đang chứa, hiện theo đúng kiểu đã chọn ở cột Type.
 - ⚠ Điểm dễ bị hỏi vặn: nếu chọn **WORD** cho một biến khai báo `db`, emulator sẽ **gộp luôn byte kế bên** vào và cho ra một số hoàn toàn khác. Byte trong bộ nhớ không hề đổi — chỉ cách diễn giải đổi. Do thực tế: `var2` để size = BYTE thì hiện **08h** (đúng giá trị khai báo), đổi sang size = WORD thì hiện **9008h** — vì nó gộp thêm byte `90h` nằm ngay sau, vốn chỉ là byte trống trong bộ nhớ.



Cùng một biến `var2`, cùng một byte trong bộ nhớ, chỉ đổi **ô size** từ `byte` sang `word`: giá trị hiển thị nhảy từ **08h** lên **9008h**. Con `90h` bị gộp thêm vào là byte trống nằm ở ô 0710Ah, không thuộc biến nào. Vì nó rơi vào nửa cao nên bị nhân với 256, làm con số chênh tới 36.864 đơn vị.

- Kết luận đúng bản chất: **bộ nhớ không có khái niệm kiểu dữ liệu**. Kiểu là do chương trình, và do cửa sổ này, áp đặt lên khi đọc.

Bài 11: Mảng, ôn ý nghĩa thanh ghi

Đính chính của thầy ghi ngay đầu phần mô tả video: 4 thanh ghi hỗ trợ truy xuất bộ nhớ (thông qua địa chỉ) là **BX, SI, DI, BP**. Còn **SP** là để hỗ trợ truy xuất **Stack**, không dùng làm con trỏ mảng. Nội dung nói trong video bị sai chỗ này.

45. Khai báo 1 biến mảng X, chứa 5 số 16 bit, ban đầu bằng 0.

- X dw 0, 0, 0, 0, 0
- Hoặc viết gọn bằng `dup`:

- X dw 5 dup(0)
- Dùng `dw` vì đề yêu cầu số **16 bit** (mỗi phần tử 2 byte). Nếu dùng `db` thì mỗi phần tử chỉ 8 bit, sai đề.
- Mảng chiếm tổng cộng $5 \times 2 = 10$ byte liên tiếp trong bộ nhớ.
- Bản chất: trong ASM **không có kiểu mảng**. `X` chỉ là **tên của ô nhớ đầu tiên**; 10 byte nằm liền nhau, còn việc coi chúng là 5 phần tử là do người lập trình tự quy ước.

46. Trong 8086 (ASM), có dạng chuỗi không?

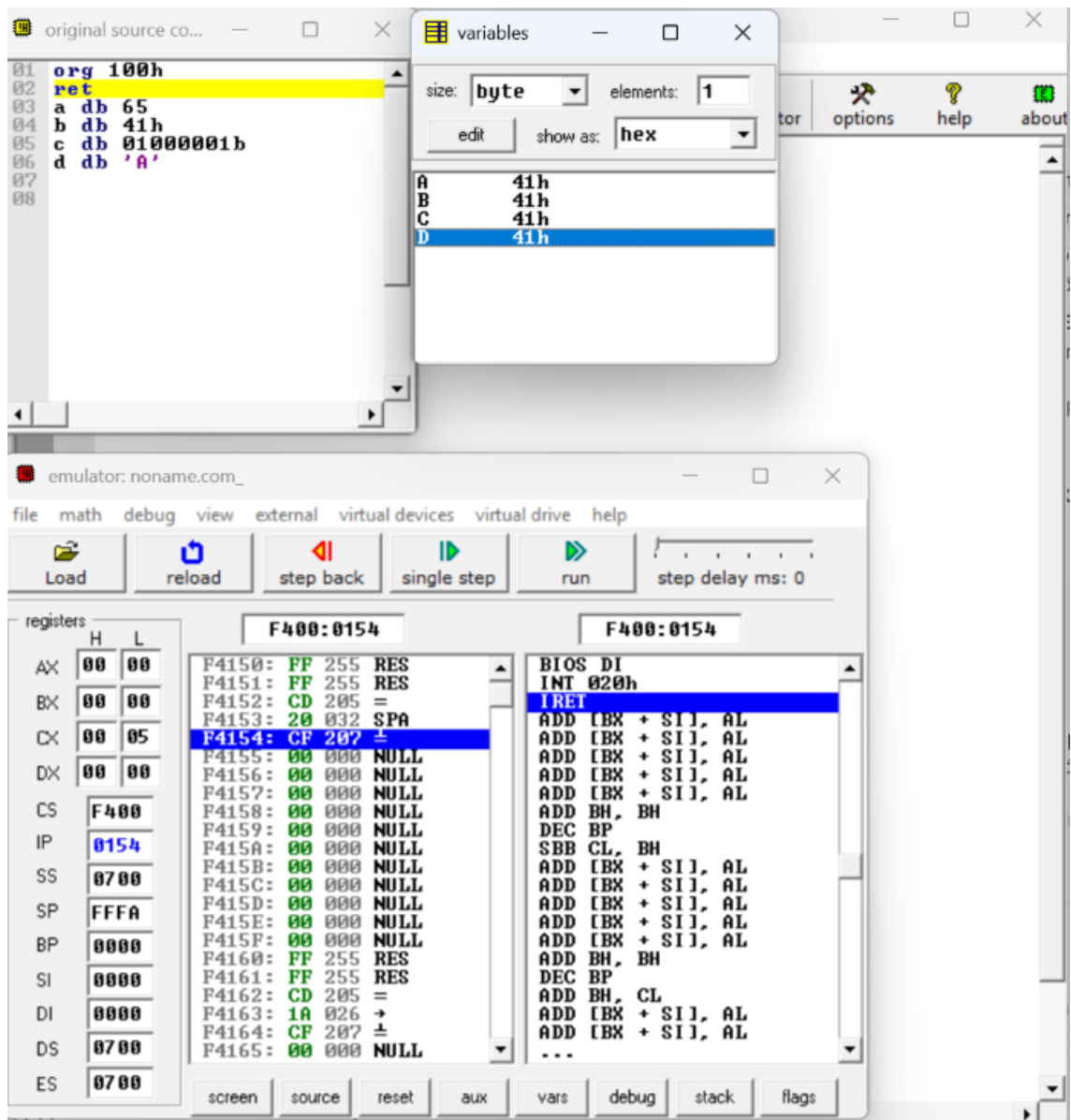
- **Không có**. ASM 8086 không có kiểu dữ liệu chuỗi.
- Cái ta quen gọi là "chuỗi" thực chất chỉ là **một dãy byte liên tiếp chứa mã ASCII**, và phải **tự quy ước chỗ kết thúc**.
- Với hàm xuất chuỗi `INT 21h / AH = 9`, quy ước kết thúc là ký tự `'$'` (24h). Thiếu dấu `'$'` thì hàm cứ in tiếp các byte rác phía sau cho tới khi tình cờ gặp một byte 24h.

47. Vậy "abcd" là gì? Ghi 'abcd' có lỗi không và có khác gì so với "abcd" không?

- `"abcd"` chỉ là **cách viết tắt cho 4 byte mã ASCII liên tiếp**: `61h 62h 63h 64h`. Assembler bung nó ra thành 4 byte, không hề tạo ra kiểu dữ liệu mới nào.
- `'abcd'` **không lỗi**. Trong ASM 8086 (cú pháp MASM và emu8086), dấu nháy đơn và nháy kép **tương đương nhau**, cho ra đúng cùng 4 byte đó.
- Đây là chỗ khác hẳn với C: bên C, `'a'` là ký tự còn `"a"` là chuỗi, hai kiểu khác nhau. Trong ASM thì cả hai chỉ là byte.
- Chỉ khác nhau khi nội dung có chứa chính dấu nháy: muốn có dấu `"` bên trong thì bọc bằng nháy đơn, và ngược lại.

48. Chạy lại Code mẫu trong video, khai báo 4 biến a, b, c, d. Mở cửa sổ Variable để xem kết quả 4 biến này, xem có giống nhau không?

- Nguyên tắc để trả lời: nếu 4 biến được khai báo **cùng một giá trị nhưng viết ở 4 hệ đếm khác nhau** (ví dụ `db 65`, `db 41h`, `db 01000001b`, `db 'A'`) thì trong cửa sổ Variables **giá trị hiện ra giống hệt nhau**.
- Lý do: assembler đã **quy đổi hết về cùng một byte** ngay lúc biên dịch. Tới lúc chạy thì trong bộ nhớ chỉ còn đúng một byte `41h`, không còn dấu vết của cách viết ban đầu.
- Đây chính là bằng chứng trực quan cho ý cốt lõi: **hệ đếm chỉ là cách viết cho người đọc**, máy luôn lưu nhị phân.
- Nếu 4 biến khai báo **khác kích thước** (`db` với `dw`) thì cột Address sẽ cho thấy khoảng cách giữa các biến khác nhau — 1 byte so với 2 byte.
- **Kết quả chạy thực tế:** khai báo `a db 65`, `b db 41h`, `c db 01000001b`, `d db "A"` rồi mở `Emulator / view / variables` — **cả bốn biến đều hiện 41h**, giống hệt nhau, không phân biệt được cách nào viết ra cách nào.



Bốn biến `a db 65`, `b db 41h`, `c db 01000001b`, `d db 'A'` — bốn cách viết khác hẳn nhau trong mã nguồn, nhưng cửa sổ Variables hiện **cả bốn đều bằng 41h**. Không còn dấu vết nào của cách viết ban đầu, vì assembler đã quy đổi hết về cùng một byte ngay lúc biên dịch.

49. Viết chương trình: khai báo 1 biến mảng X, chứa 5 số 16 bit, ban đầu bằng 0. Nhập giá trị 1, 2, 3, 4, 5 cho 5 phần tử này. Yêu cầu: dùng chỉ số của mảng là 1 con số, vd X[1].

```
org 100h
```

```

mov X[0], 1      ; phần tử thứ 1
mov X[2], 2      ; phần tử thứ 2
mov X[4], 3      ; phần tử thứ 3
mov X[6], 4      ; phần tử thứ 4
mov X[8], 5      ; phần tử thứ 5

```

```
ret
```

```
X dw 5 dup(0)
```

- ⚠ **Chỗ quan trọng nhất của câu này:** con số trong `X[...]` là **số byte lệch so với đầu mảng (offset)**, không phải số thứ tự phần tử.

- Vì mỗi phần tử là 16 bit = **2 byte**, nên phần tử thứ n nằm ở **$X[(n-1) \times 2]$** — chỉ số nhảy 0, 2, 4, 6, 8 chứ không phải 0, 1, 2, 3, 4.
- Viết **$X[1]$** là trỏ vào **byte cao của phần tử thứ nhất**, không phải phần tử thứ hai. Đây là lỗi kinh điển và gần như chắc chắn bị hỏi vặn.
- Nếu mảng khai báo bằng **db** (mỗi phần tử 1 byte) thì lúc đó chỉ số mới trùng với thứ tự phần tử: **$X[0]$** , **$X[1]$** , **$X[2]$** ...
- Cách nhớ gọn: **chỉ số trong ngoặc vuông = thứ tự phần tử $\times$ kích thước một phần tử**.

50. Các thanh ghi nào có thể sử dụng để chứa chỉ số để truy xuất đến các phần tử của mảng?
Viết lại chương trình câu 5, dùng 1 thanh ghi để chứa chỉ số mảng.

- Theo đánh chính của thầy: **BX, SI, DI, BP** — đúng 4 thanh ghi này được dùng làm con trỏ / chỉ số truy xuất bộ nhớ.
- **SP không dùng được** cho việc này. SP dành riêng cho stack, nó luôn trỏ vào đỉnh stack và bị các lệnh PUSH, POP, CALL, RET tự động thay đổi.
- Trong thực tế làm mảng, hay dùng **SI** và **DI** nhất (Source Index và Destination Index, sinh ra đúng để làm chỉ số), rồi mới tới **BX**.
- **$\triangle$ $[BP]$ mặc định lấy segment SS** chứ không phải **DS**, nên dùng **BP** để duyệt mảng dữ liệu thì phải ghi rõ segment: **$DS:[BP]$** .
- Viết lại chương trình câu 5, dùng **SI** làm chỉ số:

```
org 100h
```

```
mov si, 0
mov X[si], 1
add si, 2           ; nhảy sang phần tử kế tiếp, mỗi phần tử 2 byte
mov X[si], 2
add si, 2
mov X[si], 3
add si, 2
mov X[si], 4
add si, 2
mov X[si], 5
```

```
ret
```

```
X dw 5 dup(0)
```

- Mỗi lần **$add si, 2$** là nhảy sang phần tử kế tiếp — vẫn đúng nguyên tắc "cộng thêm kích thước một phần tử", chỉ khác là giờ con số nằm trong thanh ghi thay vì viết cứng.
- Gọn hơn nữa bằng vòng lặp, đây mới là lý do người ta dùng thanh ghi làm chỉ số:

```
org 100h
```

```
mov si, 0           ; chỉ số chạy theo byte
mov cx, 5           ; CX là bộ đếm của lệnh LOOP
mov ax, 1           ; giá trị cần gán, tăng dần
```

```
lap:
```

```
    mov X[si], ax
    add si, 2
    inc ax
    loop lap        ; CX tự giảm 1, khác 0 thì nhảy về lap
```

```
ret
```

```
X dw 5 dup(0)
```

- Ở đây thấy rõ ý nghĩa từng thanh ghi đúng như tên gọi: **SI** giữ chỉ số, **CX** làm bộ đếm cho ``LOOP`` (lệnh này tự trừ CX), **AX** giữ dữ liệu. Viết cứng ``X[0]``, ``X[2]`` ... thì không lặp được.

Bài 12: Hằng số

51. Vị trí khai báo hằng số?

- Tốt nhất đặt **sau lệnh ``ret``**, cùng chỗ với khai báo biến, cho thống nhất bố cục.
- Nhưng lý do thì **khác với biến**: hằng số khai báo bằng ``EQU`` **không chiếm ô nhớ nào cả**, nên về mặt kỹ thuật đặt ở đâu cũng chạy được — kể cả trước ``org 100h``. Nó không bao giờ bị CPU giải mã nhằm thành lệnh, vì nó không hề tồn tại trong bộ nhớ.
- Yêu cầu duy nhất của assembler: **phải khai báo trước dòng lệnh đầu tiên sử dụng nó** (assembler đọc mã nguồn từ trên xuống).
- Còn nếu hiểu "hằng số" theo nghĩa một **biến chứa giá trị không đổi** (``db``, ``dw``) thì bắt buộc phải để sau ``ret``, vì nó chiếm ô nhớ thật.

52. Khai báo 1 hằng số chứa giá trị là 150.

```
N equ 150
```

- Viết ở hệ đếm khác cho cùng giá trị cũng được, assembler quy đổi hết lúc biên dịch:

```
N equ 96h           ; hệ 16
```

```
N equ 10010110b     ; hệ 2
```

- Cách dùng trong chương trình:

```
org 100h
```

```
mov al, N           ; assembler thay N bằng số 150 ngay lúc biên dịch
```

```
ret
```

```
N equ 150
```

53. Khác biệt giữa khai báo 1 hằng số và khai báo 1 biến chứa cùng 1 giá trị là gì?

	Hằng số <code>`N equ 150`</code>	Biến <code>`N db 150`</code>
Chiếm ô nhớ?	Không — không tồn tại trong bộ nhớ	Có — 1 byte trong bộ nhớ
Tồn tại tới lúc nào?	Chỉ tới lúc biên dịch	Tồn tại cả lúc chạy
Assembler làm gì?	Thay thẳng số 150 vào mã lệnh	Sinh mã lệnh đi đọc ô nhớ
Có địa chỉ không?	Không có	Có, cả logic lẫn vật lý
Đổi được lúc chạy?	Không	Được
Xem trong Memory được?	Không, vì không có gì để xem	Được
<code>`MOV AL, N`</code> dịch ra	<code>`B0 96`</code> — 2 byte , hằng số nhúng thẳng	<code>`A0 xx xx`</code> — 3 byte , đi lấy từ ô nhớ

- Ý cốt lõi: ``equ`` **chỉ là phép thay thế văn bản lúc biên dịch**, giống hệt ``#define`` bên C. Sau khi biên dịch, cái tên ``N`` biến mất hoàn toàn và trong mã máy chỉ còn con số.
- Còn biến thì sinh ra **ô nhớ thật, có địa chỉ thật**, đọc được bằng ``Emulator / view / Memory``.
- Hệ quả thực tế: dùng ``equ`` thì chương trình **ngắn hơn và chạy nhanh hơn** (không phải đi ra bộ nhớ lấy dữ liệu), nhưng **không sửa được giá trị lúc chạy**.
- Cách kiểm chứng nhanh khi bị hỏi: viết hai chương trình, một dùng ``equ`` một dùng ``db``, rồi so **số byte của cùng một lệnh MOV** trong file ``.list`` — sẽ thấy chênh nhau đúng 1 byte.

Bài 13: Lệnh ASM chia làm 4 nhóm

54. Tập lệnh ASM của 8086 chia làm mấy nhóm lệnh? Ý nghĩa của từng nhóm lệnh?

- Chia làm **4 nhóm**:
- **Nhóm 1 — Di chuyển (sao chép) dữ liệu.** Chuyển dữ liệu giữa thanh ghi, ô nhớ và cổng vào/ra. Đặc điểm quan trọng: **không làm thay đổi thanh ghi cờ**. Gồm MOV, XCHG, PUSH, POP, LEA, IN, OUT.
- **Nhóm 2 — Số học và logic.** Thực hiện phép tính. Đặc điểm quan trọng: **có cập nhật thanh ghi cờ (Flags)** sau mỗi phép tính, và đó là cơ sở cho nhóm rẽ nhánh hoạt động. Gồm ADD, SUB, INC, DEC, MUL, DIV, CMP, AND, OR, XOR, NOT, NEG, các lệnh dịch/quay SHL, SHR, ROL, ROR.
- **Nhóm 3 — Rẽ nhánh, điều khiển chương trình.** Thay đổi luồng thực thi bằng cách **ghi giá trị mới vào IP** (và cả CS nếu nhảy xa). Gồm JMP, các lệnh nhảy có điều kiện Jxx, LOOP, CALL, RET, INT.
- **Nhóm 4 — Điều khiển hệ thống và xử lý chuỗi.** Điều khiển trực tiếp cờ và trạng thái CPU, cùng các lệnh thao tác trên dãy byte. Gồm CLC, STC, CLD, STD, CLI, STI, NOP, HLT, và nhóm chuỗi MOVSB, STOSB, LODSB, CMPSB.
- Mối liên hệ đáng nhớ giữa các nhóm: **nhóm 2 đặt cờ → nhóm 3 đọc cờ để quyết định nhảy**. Ví dụ `CMP AL, 5` (nhóm 2) đặt cờ, rồi `JE nhanh` (nhóm 3) đọc cờ ZF để quyết định có nhảy hay không. Hai nhóm này luôn đi cặp với nhau.
- Một số sách gộp nhóm 4 thành "nhóm điều khiển" và tách riêng "nhóm xử lý chuỗi" thành nhóm thứ 5. Nắm được cách chia 4 nhóm ở trên là đủ, nhưng nên biết là có cách chia khác để không bị hụt khi thầy hỏi.

55. Lập bảng mô tả ý nghĩa của từng lệnh.

- **Nhóm 1 — Di chuyển dữ liệu** (không đổi cờ)

Lệnh	Ý nghĩa
`MOV đích, nguồn`	Sao chép dữ liệu từ nguồn sang đích. Nguồn không bị mất . Hai toán hạng phải cùng kích thước, và không được cả hai cùng là ô nhớ.
`XCHG a, b`	Hoán đổi giá trị của hai toán hạng cho nhau. Làm được việc mà MOV không làm được là đổi chỗ, không cần biến trung gian.
`PUSH nguồn`	Đẩy một giá trị 16 bit vào stack. SP tự giảm 2 (stack mọc ngược xuống).
`POP đích`	Lấy giá trị 16 bit ra khỏi đỉnh stack. SP tự tăng 2. Thứ tự lấy ra ngược với thứ tự đẩy vào.
`LEA đích, ô nhớ`	Lấy địa chỉ (offset) của ô nhớ chứ không lấy giá trị. `LEA DX, msg` tương đương `MOV DX, OFFSET msg`.
`IN / OUT`	Đọc / ghi dữ liệu qua cổng vào-ra của thiết bị ngoại vi.

- **Nhóm 2 — Số học và logic** (có cập nhật cờ)

Lệnh	Ý nghĩa
`ADD đích, nguồn`	Cộng: đích = đích + nguồn.
`SUB đích, nguồn`	Trừ: đích = đích - nguồn.
`INC đích`	Tăng đích lên 1. Ngắn hơn `ADD đích, 1` về số byte.
`DEC đích`	Giảm đích đi 1.
`MUL nguồn`	Nhân không dấu . Kết quả luôn để ở chỗ cố định: nhân 8 bit thì ra AX; nhân 16 bit thì ra

	cặp DX:AX (DX giữ word cao, AX giữ word thấp).
`DIV nguồn`	Chia không dấu . Số bị chia lấy từ AX hoặc DX:AX. Chia 16 bit cho 8 bit: thương vào AL, dư vào AH .
`CMP a, b`	So sánh bằng cách trừ thử a – b nhưng không lưu kết quả , chỉ đặt cờ . Luôn đi kèm một lệnh nhảy có điều kiện ngay sau.
`AND / OR / XOR`	Phép logic theo từng bit. Hay dùng để bật, tắt, đảo bit cụ thể. `XOR AX, AX` là cách xoá AX về 0 nhanh nhất.
`NOT đích`	Đảo toàn bộ bit (bù 1).
`NEG đích`	Đổi dấu (bù 2), tức đích = 0 – đích.
`SHL / SHR`	Dịch bit trái / phải. Dịch trái 1 bit là nhân 2, dịch phải 1 bit là chia 2.
`ROL / ROR`	Quay bit trái / phải, bit rơi ra ngoài được đưa vòng lại đầu bên kia.

- **Nhóm 3 — Rẽ nhánh, điều khiển chương trình** (ghi giá trị mới vào IP)

Lệnh	Ý nghĩa
`JMP nhãn`	Nhảy vô điều kiện tới nhãn. Bản chất là ghi thẳng giá trị mới vào IP.
`JE / JZ`	Nhảy nếu bằng nhau (cờ ZF = 1). Dừng ngay sau CMP.
`JNE / JNZ`	Nhảy nếu khác nhau (ZF = 0).
`JB / JA`	Nhảy nếu nhỏ hơn / lớn hơn, dùng cho số không dấu .
`JL / JG`	Nhảy nếu nhỏ hơn / lớn hơn, dùng cho số có dấu . Nhầm cặp này với JB/JA là lỗi rất hay gặp.
`LOOP nhãn`	Tự động giảm CX đi 1 , nếu CX vẫn khác 0 thì nhảy về nhãn. Bắt buộc dùng CX làm bộ đếm, không thay bằng thanh ghi khác được.
`CALL nhãn`	Gọi chương trình con: đẩy địa chỉ trở về (IP hiện tại) vào stack rồi mới nhảy.
`RET`	Kết thúc chương trình con: lấy địa chỉ trở về từ stack nạp vào IP. Mã máy `C3`, 1 byte.
`INT n`	Gọi ngắt số n — nhờ DOS hoặc BIOS làm hộ một việc (nhập, xuất, thoát...).

- **Nhóm 4 — Điều khiển hệ thống và xử lý chuỗi**

Lệnh	Ý nghĩa
`CLC / STC`	Xoá / đặt cờ nhớ CF.
`CLD / STD`	Đặt chiều duyệt chuỗi tăng dần / giảm dần (cờ DF).
`CLI / STI`	Cấm / cho phép ngắt.

`NOP`	Không làm gì, chỉ tốn 1 byte và 1 nhịp. Dùng để chèn chỗ trống hoặc căn địa chỉ.
`HLT`	Dừng CPU cho tới khi có ngắt.
`MOVSb / MOVSw`	Chép 1 byte / 1 word từ DS:SI sang ES:DI , rồi tự tăng (hoặc giảm) cả SI và DI. Đây là chỗ thấy rõ vai trò của thanh ghi ES .
`LODSb / STOSb`	Nạp byte từ DS:SI vào AL / ghi AL ra ES:DI, kèm tự tăng chỉ số.
`CMPSb`	So sánh byte tại DS:SI với byte tại ES:DI và đặt cờ.
`REP`	Tiền tố lặp, ghép trước lệnh chuỗi, lặp lại đúng CX lần.

Bài 14: Lệnh MOV và các loại toán hạng (operand)

56. Có bao nhiêu loại toán hạng (operand)?

- Có 4 loại:
 - o **Thanh ghi (register)** — AX, BL, SI, DS... Toán hạng nằm ngay trong CPU, truy cập nhanh nhất.
 - o **Hằng số / giá trị tức thời (immediate)** — 1Fh, 20, 'A'. Giá trị nằm **ngay trong mã lệnh**, không phải đi lấy ở đâu cả. Chỉ làm **nguồn**, không bao giờ làm đích.
 - o **Ô nhớ, địa chỉ trực tiếp (direct memory)** — viết bằng tên biến, ví dụ `var`. Assembler thay tên bằng **địa chỉ số cố định** nhúng vào mã lệnh.
 - o **Ô nhớ, địa chỉ gián tiếp (indirect memory)** — `[BX]`, `[SI]`, `[DI]`, `[BP]`, `[BX+SI]`, `var[SI]`. Địa chỉ nằm **trong thanh ghi**, nên đổi được lúc chạy — đây là thứ cho phép duyệt mảng.
- Có sách gộp loại 3 và 4 lại thành một loại chung là **"bộ nhớ (memory)"**, khi đó chỉ còn **3 loại**: register, immediate, memory. Nói được cả hai cách chia thì an toàn khi bị hỏi.
- Điểm mấu chốt phân biệt: **hằng số nằm trong mã lệnh, ô nhớ nằm ngoài mã lệnh (phải đi lấy qua bus), thanh ghi nằm trong CPU**. Ba chỗ khác nhau, tốc độ khác nhau.

57. Các toán hạng: AX, DS, BL, SS, 1Fh, 20, variable, [DI], SI, [SI] thuộc loại toán hạng nào?

Toán hạng	Loại	Ghi chú
`AX`	Thanh ghi đa dụng, 16 bit	Tách được thành AH và AL
`DS`	Thanh ghi đoạn (segment register)	Không nạp hằng số trực tiếp vào được
`BL`	Thanh ghi đa dụng, 8 bit	Là nửa thấp của BX
`SS`	Thanh ghi đoạn	Trỏ tới segment của stack
`1Fh`	Hằng số (immediate), hệ 16	Bằng 31 ở hệ 10
`20`	Hằng số (immediate), hệ 10	Bằng 14h ở hệ 16
`variable`	Ô nhớ, địa chỉ trực tiếp	Tên biến, assembler thay bằng địa chỉ
`[DI]`	Ô nhớ, địa chỉ gián tiếp	Segment mặc định là DS
`SI`	Thanh ghi chỉ số , 16 bit	Không có ngoặc vuông nên là thanh ghi , không phải ô nhớ
`[SI]`	Ô nhớ, địa chỉ gián tiếp	Segment mặc định là DS

- ⚠ Chỗ chắc chắn bị hỏi vặn: **`SI` và `[SI]` khác hẳn nhau.**
 - o `MOV AX, SI` — copy **nội dung của thanh ghi SI** (một con số) vào AX.

- `MOV AX, [SI]` — coi con số trong SI là **địa chỉ**, đi tới ô nhớ đó lấy giá trị mang về AX.
- Dấu ngoặc vuông đọc là "**nội dung của ô nhớ tại địa chỉ này**". Thêm hay bớt ngoặc là đổi hẳn ý nghĩa câu lệnh.

58. Tìm câu lệnh SAI trong các lệnh sau, và giải thích vì sao?

```
ORG 100H
MOV AX, 0B800h
MOV DS, AX
MOV BX, CL
MOV DS, 123h
MOV VAR, CX
RET
Var DB 13
```

- Có **3 câu sai**:
- `**MOV BX, CL**` — **SAI: lệch kích thước toán hạng**. BX là **16 bit**, CL là **8 bit**. Lệnh MOV bắt buộc hai toán hạng **cùng kích thước**. Sửa thành `**MOV BX, CX**` (16 bit cả hai) hoặc `**MOV BL, CL**` (8 bit cả hai).
- `**MOV DS, 123h**` — **SAI: không được nạp hằng số trực tiếp vào thanh ghi đoạn**. Trong tập lệnh 8086 **không tồn tại mã máy** cho phép làm việc đó. Phải đi vòng qua một thanh ghi đa dụng:

```
MOV AX, 123h
MOV DS, AX
```

- Bằng chứng nằm ngay trong chính chương trình này: hai dòng đầu `**MOV AX, 0B800h**` rồi `**MOV DS, AX**` chính là làm đúng cách đó.
- `**MOV VAR, CX**` — **SAI: lệch kích thước toán hạng**. `**Var DB 13**` chỉ là **1 byte**, còn CX là **16 bit**. Sửa thành `**MOV VAR, CL**`, hoặc đổi khai báo thành `**Var DW 13**` nếu thật sự cần biến 16 bit.
- Ba câu còn lại (`**MOV AX, 0B800h**`, `**MOV DS, AX**`, `**RET**`) đều **đúng**.
- Các quy tắc của MOV nên thuộc, vì mọi câu sai đều rơi vào một trong số này:
 - Hai toán hạng phải **cùng kích thước**.
 - Không MOV thẳng **ô nhớ sang ô nhớ** — `**MOV var1, var2**` sai, phải qua thanh ghi trung gian.
 - Không MOV thẳng **hằng số vào thanh ghi đoạn** — phải qua thanh ghi đa dụng.
 - Không MOV **thanh ghi đoạn sang thanh ghi đoạn** — `**MOV DS, ES**` sai.
 - Không được ghi vào **CS**, và không tồn tại lệnh `**MOV IP, ...**`.
 - Hằng số **không bao giờ làm toán hạng đích** — `**MOV 5, AX**` sai.

59. Viết chương trình gán các thanh ghi AX = 50, BH = 30, CX = 20, DL = 300h, DS = 800h. Chạy chương trình và xem kết quả các thanh ghi trên giao diện emulator.

- Đề này có **hai cái bẫy**, phải nói ra trước khi viết chương trình:
- **Bẫy 1** — `**DL = 300h**` là **không thể**. DL là thanh ghi **8 bit**, chứa tối đa FFh tức 255. Mà 300h = **768**, vượt xa. Assembler sẽ báo lỗi ngay lúc biên dịch. Đây đúng là bẫy đã gặp ở `**MOV DH, 300**` (Bài 5) và `**MOV DL, 1234**` (Bài 3).
- Muốn giữ giá trị 300h thì phải dùng thanh ghi **16 bit**, tức `**MOV DX, 300h**` — khi đó DH = 03h và DL = 00h.
- **Bẫy 2** — `**DS = 800h**` **không gán trực tiếp được**, vì DS là thanh ghi đoạn. Phải mượn một thanh ghi đa dụng làm trung gian.
- Chương trình đúng:

```
org 100h
```

```
mov ax, 800h      ; phải làm DS TRƯỚC, vì bước này mượn AX
mov ds, ax
```

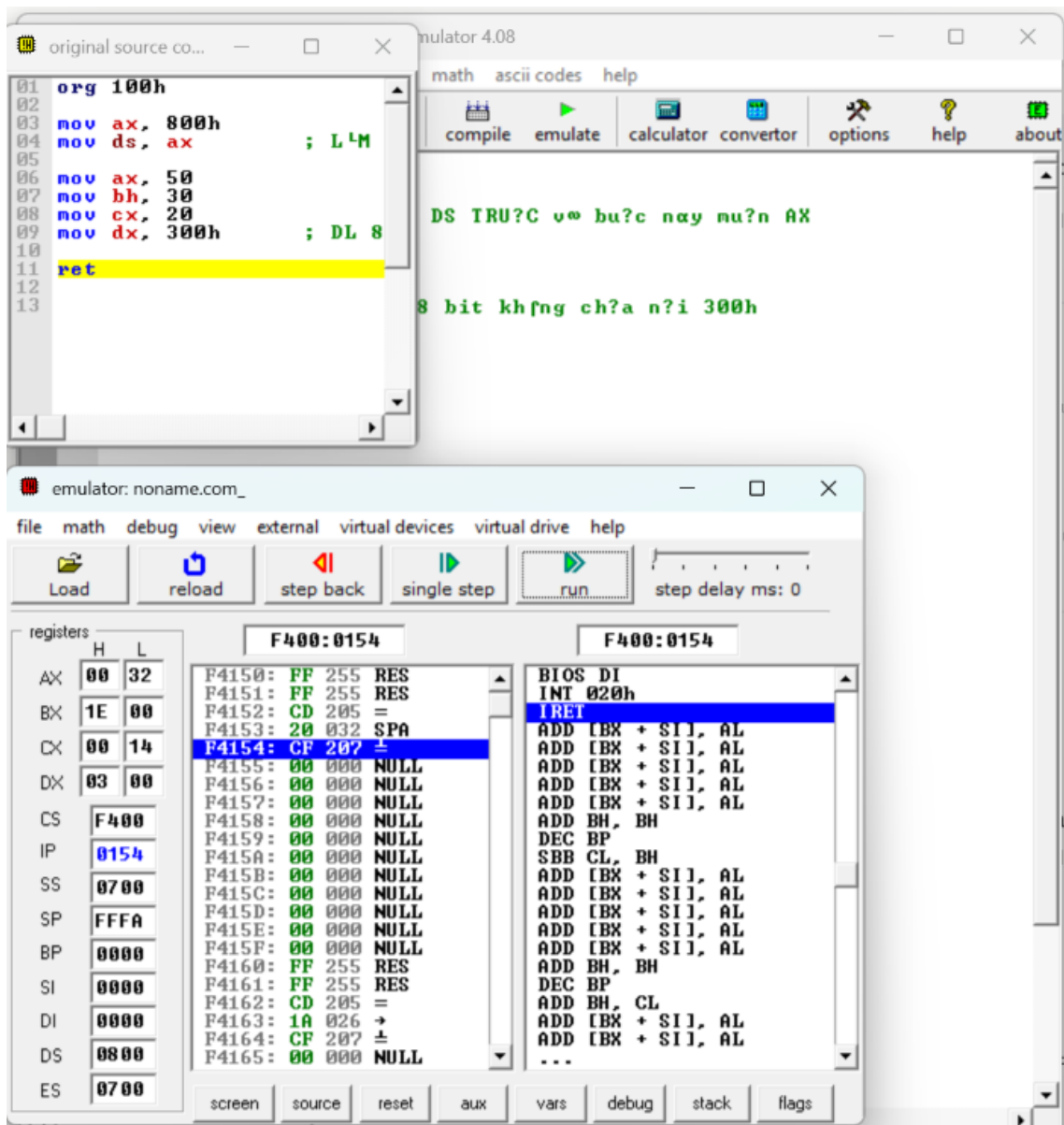
```

mov ax, 50          ; giờ mới gán AX = 50
mov bh, 30
mov cx, 20
mov dx, 300h        ; DL 8 bit không chứa nổi 300h, phải dùng DX

```

```
ret
```

- **⚠ Thứ tự rất quan trọng.** Vì phải mượn AX để nạp DS, nên bắt buộc **gán DS trước rồi mới gán AX = 50**. Nếu làm ngược lại, tới lúc `mov ax, 800h` thì AX không còn bằng 50 nữa — chụp hình sẽ thấy AX = 800h và mất điểm. Đây là chỗ mất điểm dễ nhất của cả bài.
- Kết quả phải thấy trên bảng registers: **AX = 0032h (50), BH = 1Eh (30), CX = 0014h (20), DX = 0300h, DS = 0800h.**
- **Đã chạy và xác nhận:** bảng registers cho ra đúng `AX = 0032h`, `BH = 1Eh`, `CX = 0014h`, `DX = 0300h`, `DS = 0800h`.



Bảng registers sau khi chạy: **AX = 0032h (50), BH = 1Eh (30), CX = 0014h (20), DX = 0300h, DS = 0800h**. Chú ý DX: nửa cao DH = 03h còn **DL = 00h** — tức yêu cầu "DL = 300h" của đề là bất khả thi, chỉ DX 16 bit mới chứa nổi. Và AX vẫn giữ được 50 vì đã nạp DS trước rồi mới gán AX.

- Lưu ý khi chụp hình: sau khi `MOV DS, AX`, DS không còn bằng 0700h nữa nên **cửa sổ Memory và Variables sẽ trở sang vùng khác** — điều này bình thường, không phải lỗi.
- Về `MOV DX, 300h`: kết quả `DX = 0300h` nghĩa là **DH = 03h và DL = 00h**. Tức là **DL vẫn không hề bằng 300h** — không có cách nào nhét 300h vào một thanh ghi 8 bit. Đề bài này cố tình ra một yêu cầu bất khả thi để xem mình có phát hiện ra không.

Bài 15: Nhập, xuất kí tự và xuất chuỗi

60. Viết ứng dụng chuyển kí tự hoa sang thường (xuất thông báo, nhập 1 kí tự hoa, xuất kí tự thường tương ứng, có dừng màn hình chờ nhấn phím).

- Chương trình hoàn chỉnh:

```
org 100h

; --- 1. Xuất câu thông báo ---
mov ah, 9
mov dx, offset msg1
int 21h

; --- 2. Nhập 1 kí tự chữ hoa ---
mov ah, 1
int 21h                ; mã ASCII của phím vừa nhấn nằm trong AL

; --- 3. Đổi hoa sang thường ---
add al, 20h            ; 'A' = 41h -> 'a' = 61h, chênh đúng 20h
mov kq, al             ; lưu kết quả vào biến

; --- 4. Xuống dòng ---
mov ah, 2
mov dl, 13             ; CR: đưa con trỏ về đầu dòng
int 21h
mov dl, 10             ; LF: xuống dòng mới
int 21h

; --- 5. Xuất câu thông báo thứ hai ---
mov ah, 9
mov dx, offset msg2
int 21h

; --- 6. Xuất kí tự thường ---
mov ah, 2
mov dl, kq
int 21h

; --- 7. Dừng màn hình, chờ nhấn phím bất kì ---
mov ah, 0
int 16h

ret

msg1 db "Nhap 1 ki tu chu hoa: $"
msg2 db "Ki tu thuong tuong ung: $"
kq    db ?
```

- **Các ngắt đã dùng và quy ước của từng cái:**

Ngắt	Chức năng	Tham số
`INT 21h / AH = 9`	Xuất một chuỗi	Địa chỉ chuỗi để trong DX , chuỗi phải kết thúc bằng `\$`
`INT 21h / AH = 1`	Nhập một kí tự	Mã ASCII trả về trong AL , đồng thời tự hiện kí tự lên màn hình
`INT 21h / AH = 2`	Xuất một kí tự	Kí tự cần xuất để trong DL
`INT 16h / AH = 0`	Dừng, chờ nhấn phím bất kì	Không cần tham số

- **Giải thích những chỗ dễ sai:**
- **Vì sao là `mov dx, offset msg1` chứ không phải `mov dx, msg1`?** Vì `INT 21h/AH=9` cần **địa chỉ** của chuỗi, không cần nội dung. `offset msg1` lấy offset của biến; còn `mov dx, msg1` sẽ lấy **2 byte đầu của nội dung chuỗi** đưa vào DX, ra kết quả hoàn toàn sai. Viết `lea dx, msg1` cũng cho kết quả tương đương.
- **Vì sao chuỗi phải có `\$` ở cuối?** Vì trong ASM **không có kiểu chuỗi** (Bài 11 câu 2) — chuỗi chỉ là dãy byte ASCII liên tiếp, bản thân nó không mang thông tin về độ dài. `\$` (mã 24h) là **quy ước chỗ dừng** của hàm AH=9. Thiếu `\$` thì hàm cứ in tiếp các byte rác phía sau cho tới khi tình cờ gặp một byte 24h.
- **Vì sao `add al, 20h` lại đổi hoa thành thường?** Vì trong bảng ASCII, chữ thường luôn lớn hơn chữ hoa tương ứng **đúng 32 = 20h**: `A` = 41h thì `a` = 61h, `B` = 42h thì `b` = 62h. Quy luật này đúng cho toàn bộ 26 chữ cái.
- Chiều ngược lại (thường sang hoa) thì **trừ 20h**. Cách khác cho cùng kết quả: `or al, 20h` để bật bit thứ 5, và `and al, 0DFh` để tắt bit đó — vì bản chất hoa và thường chỉ khác nhau **đúng một bit**.
- **Vì sao phải xuất cả 13 và 10?** `13` (CR, Carriage Return) chỉ đưa con trỏ **về đầu dòng**, `10` (LF, Line Feed) mới **xuống dòng mới**. Thiếu LF thì dòng sau **đề lên** dòng trước; thiếu CR thì chữ bị lệch thành hình bậc thang. Phải có đủ cả hai.
- **Vì sao biến đặt sau `ret`?** Vì CPU không phân biệt được lệnh với dữ liệu (Bài 7 câu 2, Bài 10 câu 7). Đặt `msg1`, `msg2`, `kq` trước phần lệnh thì CPU sẽ giải mã byte ASCII như mã lệnh và chương trình hỏng ngay.
- **Vì sao cần `INT 16h / AH = 0` ở cuối?** Vì `ret` kết thúc chương trình ngay lập tức, cửa sổ emulator screen sẽ đóng hoặc bị xoá trước khi kịp nhìn kết quả. Lệnh này chặn lại, đợi người dùng nhấn một phím bất kì rồi mới cho chạy tiếp tới `ret`.
- **Về yêu cầu "Enter thì chương trình xuất ra":** `INT 21h/AH=1` trả về **ngay khi nhấn 1 phím**, không đợi Enter. Nên phím Enter mà người dùng nhấn sau đó sẽ còn nằm lại trong bộ đệm bàn phím. Muốn đúng nghĩa "nhấn Enter rồi mới xuất" thì thêm một lần đọc nữa để nuốt kí tự Enter:

```

mov ah, 1
int 21h           ; đọc kí tự hoa vào AL
mov kq, al       ; cất ngay, vì lần đọc sau sẽ ghi đè AL
add kq, 20h

```

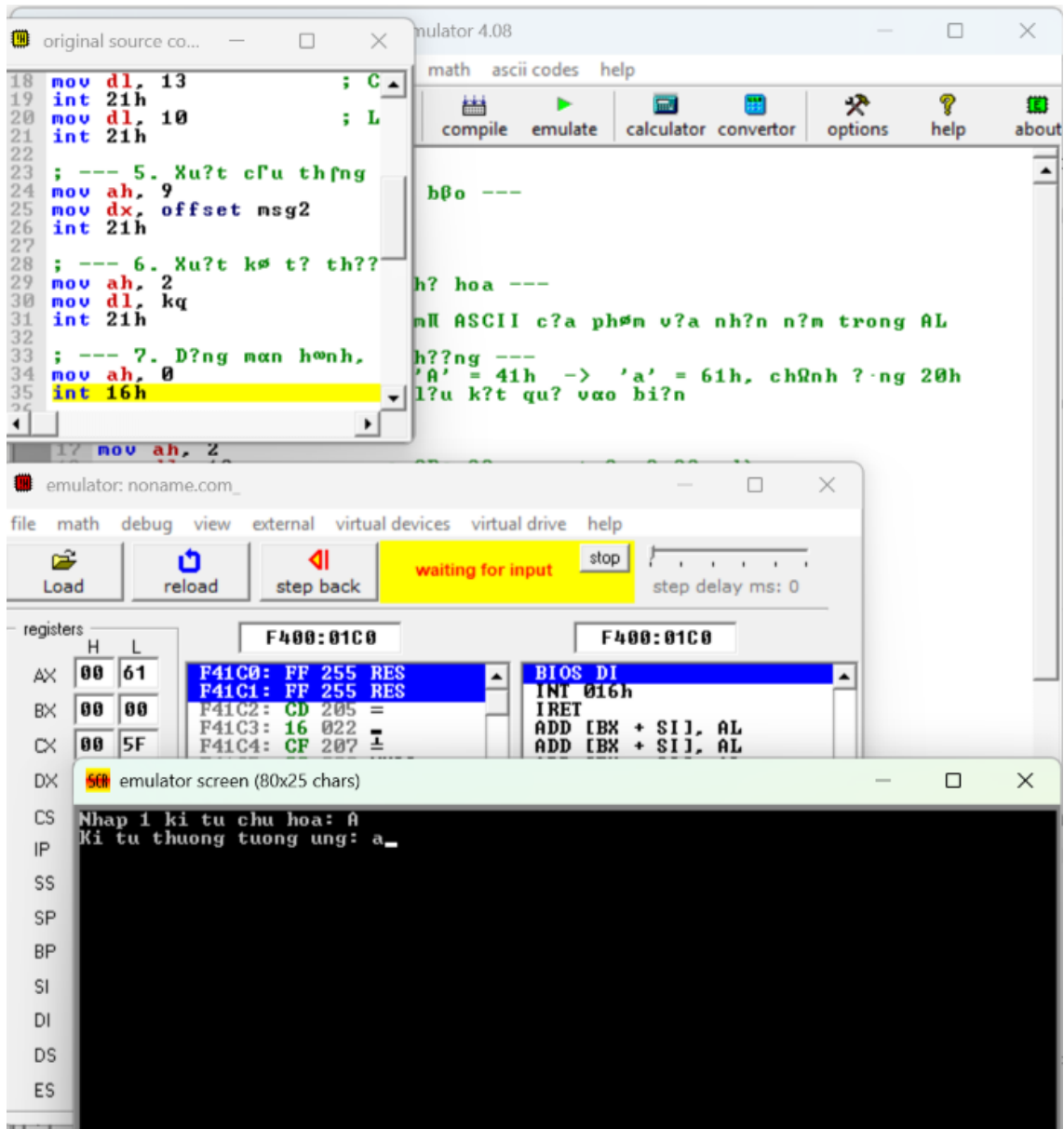
```

mov ah, 1
int 21h           ; đọc và bỏ đi kí tự Enter

```

- ⚠ Nếu làm theo cách này thì **phải cất AL vào biến trước**, vì lần `int 21h` thứ hai sẽ ghi đè lên AL. Đây là lỗi rất dễ mắc.
- **Muốn kiểm tra người dùng có nhập đúng chữ hoa không** thì thêm đoạn so sánh trước khi cộng — không bắt buộc theo đề, nhưng là điểm cộng nếu bị hỏi:

- cmp al, 'A'
 jb sai ; nhỏ hơn 'A' thì không phải chữ hoa
 cmp al, 'Z'
 ja sai ; lớn hơn 'Z' thì cũng không phải
- Ở đây `CMP` đặt cờ (nhóm 2) rồi `JB` / `JA` đọc cờ để nhảy (nhóm 3) — đúng cặp đôi đã nói ở Bài 13.
 - **Kết quả chạy thực tế:** màn hình emulator screen hiện đúng hai dòng — `Nhập 1 ki tu chu hoa: A` rồi xuống dòng `Ki tu thuong tuong ung: a`. Thanh ghi **AL = 61h**, đúng bằng mã ASCII của kí tự `a`. Chương trình dừng ở `int 16h` chờ nhấn phím, emulator báo trạng thái "waiting for input".



Cửa sổ **emulator screen** sau khi nhập chữ `A` : hiện đủ hai dòng thông báo và kí tự thường `a` tương ứng. Thanh ghi **AL = 61h** đúng bằng mã ASCII của `a`. Emulator đang báo "waiting for input" vì chương trình dừng ở `int 16h` chờ nhấn phím, đúng yêu cầu đề bài.